CHAPTER -1: INTRODUCTION

1. Introduction:

1.1. General: Real-time sign language detection is an emerging application of artificial intelligence and computer vision that aims to facilitate effective communication between individuals with hearing and speech impairments and the general population. Sign language is one of the primary modes of communication for deaf and mute individuals; however, the lack of awareness and understanding among most people creates a significant communication gap in everyday interactions.

The objective of this project is to design and develop a real-time system capable of recognizing hand gestures corresponding to sign language and converting them into meaningful text and speech output. The system uses a webcam to capture live video input, detects the hand region, and processes the image using a trained machine learning model. The recognized gestures are then translated into words, which are displayed on the screen and also converted into voice output using text-to-speech technology.

This project integrates multiple technologies such as image processing, machine learning, and speech synthesis to create a practical and user-friendly solution. The system performs real-time hand detection, extracts relevant features, and ensures uniform input through image preprocessing techniques. A trained model then classifies the gestures into predefined categories, enabling accurate recognition of different signs. Additionally, the system supports sentence formation by combining multiple detected gestures, thereby improving its usability in real-world communication scenarios. The proposed system has significant applications in areas such as healthcare, education, and public services,

1.2. Problem Statement: Despite significant advancements in artificial intelligence and computer vision, real-time sign language detection systems still face several practical challenges that limit their performance in real-world environments. Existing approaches often struggle with variations in lighting conditions, background noise, hand orientation, and differences in user behavior, which can reduce the overall accuracy and reliability of gesture recognition systems.

The central problem addressed in this project is: **How can a real time sign language detection system be designed recommender and evaluated to achieve accurate gesture accuracy while reducing performance degradation in cold-start and sparse-data scenarios?**

This problem has multiple technical dimensions:

1.2.1. Variability of Input Data: There can be variability in hand gestures among users because of differences in hand shape, size, and orientation, leading to inaccurate recognition.

1.2.2. Instantaneous Real-Time Processing: The system has to process the incoming data in real time; therefore, it needs efficient algorithms that can maintain high performance.

1.2.3. Presence of Noise in Environment: Variations in environmental factors such as lighting and background noise may adversely affect the accuracy of hand gesture detection.

1.2.4. Limited Set of Hand Gestures: There will always be some limitation on the number of gestures used for training, thus restricting the system vocabulary.

1.2.5. Model Complexity and Accuracy: Model Complexity and Accuracy: categories, titles) to construct embeddings that sidestep the limitations of pure collaborative filtering.

1.3. Objectives of the Project:

The project pursues the following objectives:

1.3.1. Design and Implement Real-Time Sign Language Detection System: To develop a system that captures hand gestures through a webcam and accurately recognizes them using computer vision and machine learning techniques.

1.3.2. Dataset Preparation: To collect, preprocess, and organize a dataset of hand gesture images for different sign language symbols, ensuring consistency in size, background, and orientation for effective model training.

1.3.3. Gesture Recognition using Trained Model: To train a machine learning model capable of classifying hand gestures into predefined categories with high accuracy and reliability.

1.3.4. Real-Time Prediction and Display: To implement real-time prediction of gestures and display the recognized sign as text on the screen without noticeable delay.

1.3.5. Voice Output Integration: To convert the detected text output into speech using text-to-speech technology, enabling audible communication for better interaction.

1.3.6. Sentence Formation: To combine multiple recognized gestures into meaningful sentences, improving the system's usability in real-world communication scenarios.

These objectives ensure the work contributes both methodological insight (what works and why) and implementation value (a usable recommendation pipeline).

1.4. Scope of the Project: This project is scoped to the end-to-end development and real-time implementation of a sign language detection system using computer

vision and machine learning techniques. The system focuses on recognizing hand gestures through a webcam and converting them into text and speech output for effective communication.

The scope includes:

1.4.1. Data Preprocessing: Collection and preprocessing of hand gesture images, including resizing, normalization, and background standardization to create model-ready input data.

1.4.2. Model Training: Training of a machine learning model using the prepared dataset to accurately classify different sign language gestures.

1.4.3. Real-Time Hand Detection: Implementation of hand detection using computer vision techniques to identify and isolate the hand region from live video input.

1.4.4. Gesture Recognition and Prediction: Real-time prediction of gestures using the trained model and displaying the corresponding output as text on the screen.

1.4.5. Voice Output Integration: Conversion of recognized text into speech using text-to-speech technology to enable audio-based communication.

This project does not include:

1.4.6. Advanced Deep Learning Architectures:

1.4.7. Real-time stream learning or continuous production retraining infrastructure.

1.4.8. Deep neural recommendation architectures requiring large-scale distributed training.

1.4.9. Full user-interface design as a primary deliverable (the focus remains on recommendation engine quality and evaluation).

Therefore, the scope is intentionally bounded to maintain scientific rigor, reproducibility, and engineering feasibility within an academic project timeline.

1.5. Motivation: The motivation of this project is both practical and academic. From a practical point of view, communication is a fundamental need in daily life, yet individuals with hearing and speech impairments often face significant challenges in expressing themselves. Most people are not familiar with sign language, which creates a communication gap in real-world situations such as hospitals, educational institutions, and public services. Therefore, developing a system that can translate sign language into text and speech in real time is not just a technical solution but a socially impactful necessity.

From an academic perspective, real-time sign language detection provides an interesting area of study as it involves the integration of multiple domains such as computer vision, machine learning, and human-computer interaction. It allows us to explore how image processing techniques and trained models work together to recognize gestures accurately. Additionally, analyzing system performance under different conditions—such as lighting variations, background noise, and gesture diversity—adds depth to the learning experience and enhances understanding of real-world AI challenges.

From an engineering standpoint, this project offers valuable hands-on experience in building an end-to-end system, including data collection, preprocessing, model training, real-time implementation, and output generation. It also involves integrating additional features such as text-to-speech conversion and sentence formation, which improves system usability. These skills are essential for developing practical AI-based applications and provide a strong foundation for future work in computer vision and intelligent systems.

CHAPTER -2: FEASIBILITY

2. Feasibility Study

2.1. Technical Feasibility: The proposed Real-Time Sign Language Detection System is technically feasible as it is built using well-established and widely used technologies in the field of artificial intelligence and computer vision. The system is developed using Python, along with libraries such as OpenCV for image processing, TensorFlow/Keras for model training, and cvzone for efficient hand tracking. These tools provide reliable support for real-time image capture, processing, and gesture recognition. From a system design perspective, the implementation is feasible because it follows a clear and modular workflow, including data collection, preprocessing, model training, and real-time prediction. The trained model is capable of classifying hand gestures based on predefined categories, and the integration of text-to-speech functionality allows conversion of recognized gestures into voice output.The system operates using standard hardware such as a laptop with a webcam, and does not require specialized devices or high-end computational resources. Since all components are compatible and readily available, the technical implementation can be successfully completed within the scope of an academic project.

2.2. Economic (Cost) Feasibility: The project is economically feasible as it requires minimal financial investment. All the software tools and libraries used, such as Python, OpenCV, TensorFlow, and text-to-speech modules, are open source and freely available. This eliminates any licensing costs associated with development.The system can be implemented using existing hardware resources like a personal computer or laptop with a built-in webcam, which further reduces expenses. There is no need for additional hardware or paid datasets, as the dataset for training can be collected manually or generated during the development phase.

The primary costs involved are indirect, including time spent on development, testing, and debugging. However, these costs are manageable within a student project environment. Therefore, the project provides a high value-to-cost ratio by delivering a functional system without significant financial burden.

2.3. Time Feasibility: The project is time-feasible within a typical academic schedule, as it can be divided into well-defined stages such as data collection, preprocessing, model training, system integration, and testing. Each phase has clear objectives and deliverables, allowing systematic progress throughout the development process.The modular nature of the system allows different components to be developed and tested independently, which reduces overall development time. For example, dataset preparation and model training can be completed first, followed by integration with real-time detection and voice output.

Potential time constraints may arise during model training and performance tuning, especially when improving accuracy. However, these challenges can be managed by limiting the dataset size and focusing on a predefined set of gestures. With proper planning and time management, the project can be completed successfully within the given timeframe.

2.4. Operational Feasibility: The system is operationally feasible as it is designed to function in real-time using a simple and user-friendly interface. Users only need to show hand gestures in front of a webcam, and the system automatically detects, processes, and converts them into text and speech output.The system can be easily operated without requiring advanced technical knowledge, making it accessible for general users. It can be applied in practical environments such as classrooms, hospitals, and public service areas to assist communication with deaf and mute individuals.Additionally, the system is maintainable and can be updated with new gestures or improved models without major changes to the overall architecture. While advanced features like continuous learning or mobile deployment are not included in the current scope, the system provides a

strong foundation for future enhancements. Overall, the project demonstrates strong operational feasibility as a functional and practical solution for real-world communication challenges.

CHAPTER -3: SYSTEM DESIGN

3. System Design:

3.1. Overall System Architecture:

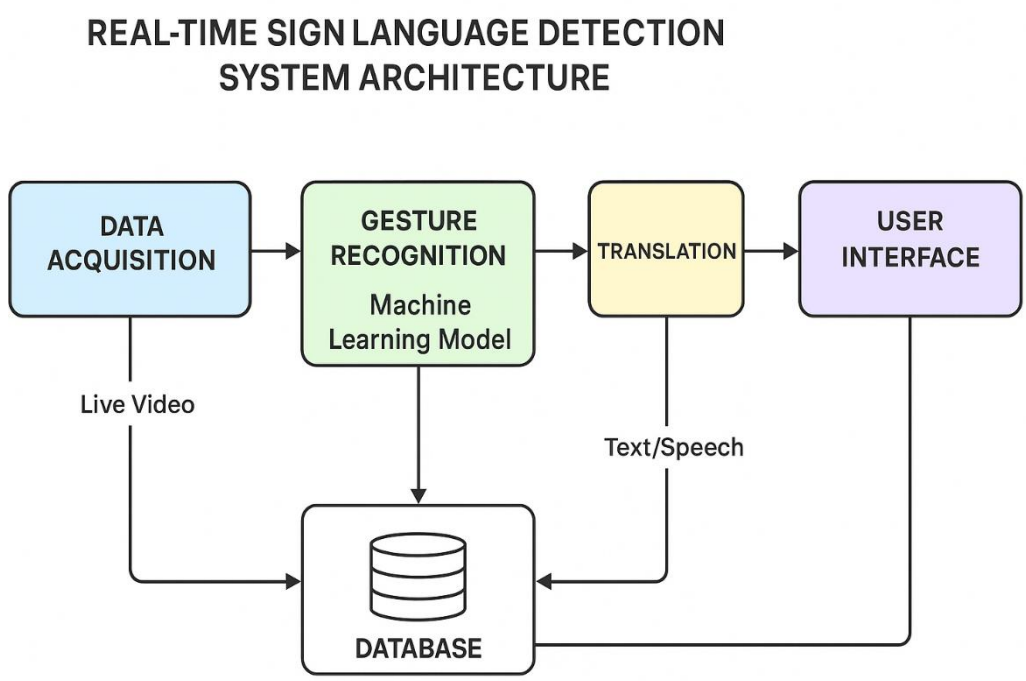


Figure 3.1: Overall Real Time Sign Language Detection System

The proposed system follows a real-time processing architecture with a clear separation between data preparation (offline) and gesture recognition (online). In the offline phase, hand gesture images are collected and preprocessed into a structured dataset. This dataset is then used to train a machine learning model capable of recognizing different sign language gestures. The trained model and

corresponding label mappings are stored as artifacts, which are later used during real-time prediction.

The online phase is responsible for real-time execution of the system. A webcam is used to capture live video input, from which hand regions are detected using computer vision techniques. The detected hand image is preprocessed (resized, normalized, and centered) to match the input format required by the trained model. The processed image is then passed to the model, which predicts the corresponding gesture label.The system further integrates text-to-speech functionality, where the predicted label is converted into voice output, enabling audio-based communication. Additionally, multiple detected gestures are combined to form meaningful sentences, improving usability in real-world scenarios.

3.2. Data Flow Diagram (DFD):

At a high level, the data flow has two coordinated paths:

3.2.1. Training Data Flow: Raw hand gesture images → image preprocessing (cropping, resizing, normalization) → structured dataset creation → model training using machine learning algorithms → generation of trained model file (keras_model.h5) and label file (labels.txt).

3.2.2. Serving Data Flow: Live webcam input → hand detection using computer vision → image cropping and preprocessing → input to trained model → gesture prediction → text output → text-to-speech conversion → display and audio response.

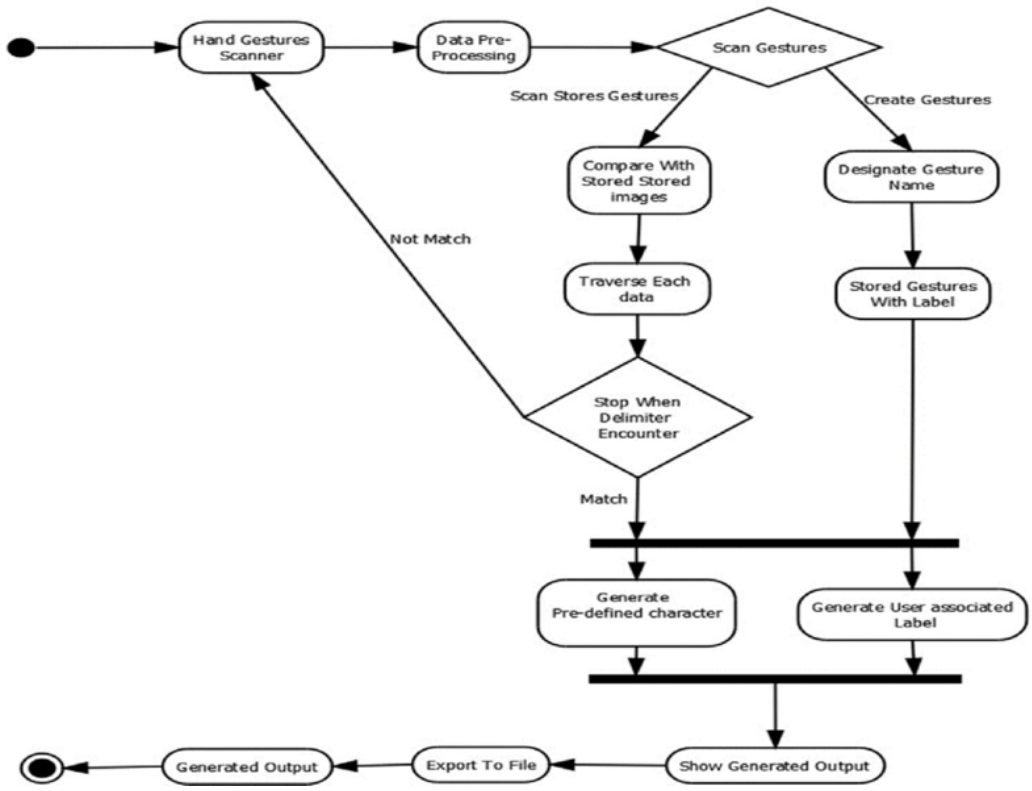


Figure 3.2: Data Flow Diagram

3.3. ER Diagram:

The logical entity relationship model is centered on four primary entities:

3.3.1. User(user_id): Represents the system user who interacts with the application by showing hand gestures in front of the camera.

3.3.2. Item/Product(item_id, title, brand, category, description, image_url): Represents different hand gestures/signs such as Hello, Yes, No, Thank You, etc., which are used as input for the system.

3.3.3. Interaction(user_id, item_id, event_type, event_value, timestamp): Represents the dataset of images used for training the model. Each image is associated with a specific gesture category.

The core relationships are defined as follows: a single user can have multiple interactions, and a single item can also be associated with multiple interactions. Each interaction links exactly one user with one item, forming a many-to-one relationship from Interaction to both User and Item.

Although the system does not use a traditional relational database, the above structure represents a logical data model equivalent to an ER schema, where data is maintained in structured file formats.

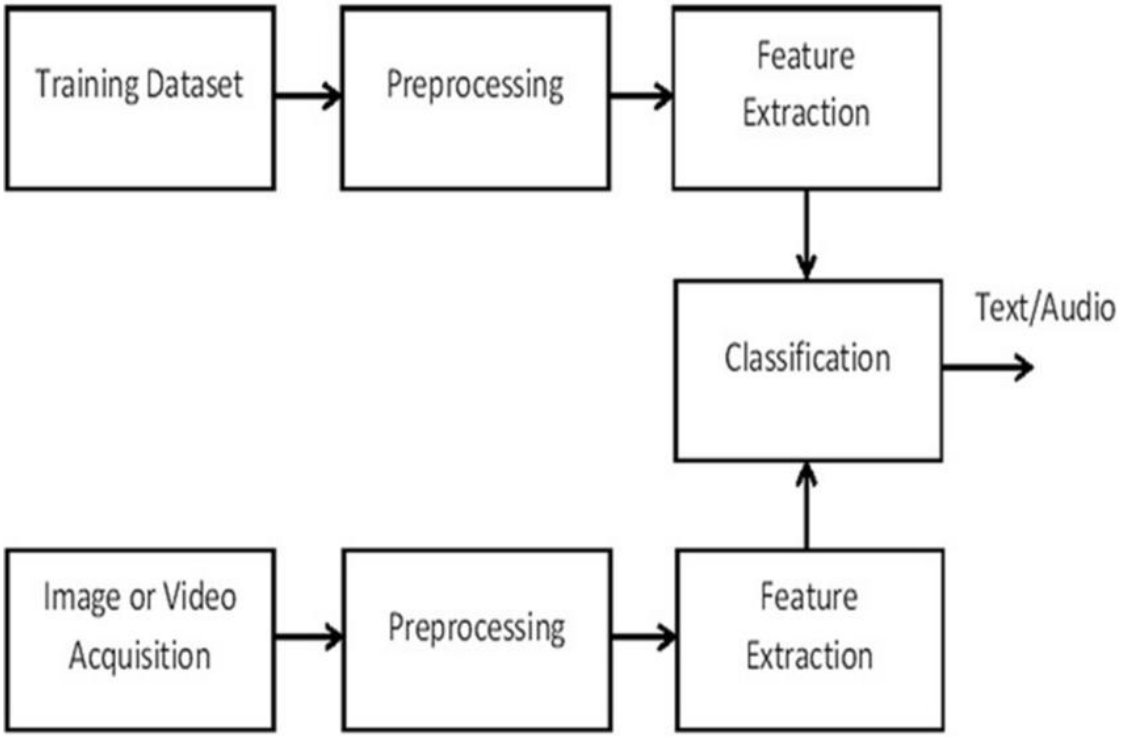


Figure 3.2: Data Flow Diagram

3.4. Database Design: The current implementation of the Real-Time Sign Language Detection System follows a file-based storage approach instead of using a traditional relational database. Structured image folders, model files, and label mappings are used as persistent storage for both training data and real-time prediction.

Key storage components include:

3.4.1. Dataset Folder (Data): Contains categorized image data for each gesture class (e.g., Hello, Yes, No, Thank You). Each folder represents a specific gesture and stores multiple training images captured using a webcam.

3.4.2. Image Files: Individual image samples captured during data collection. These images are preprocessed and used to train the machine learning model.

3.4.3. Trained Model File (keras_model.h5): Stores the trained machine learning model, which is used during real-time prediction to classify hand gestures.

3.4.4. Labels File (labels.txt): Contains the mapping of class indices to gesture names, allowing the system to interpret model predictions into readable text output.

3.4.5. Runtime Variables (In-Memory Data): Stores temporary data such as detected gestures, prediction confidence, and sentence formation during system execution.

From a design perspective, the dataset folders provide structured input for model training, while the trained model and label files act as the core components for real-time inference. This file-based approach ensures simplicity, easy management, and efficient execution without requiring complex database systems. It is well-suited for an academic project where the focus is on system functionality and performance rather than large-scale data management.

3.5. Design Constraints: Several constraints influence the design and implementation of the Real-Time Sign Language Detection System.

3.5.1. Data Constraints: The accuracy of gesture recognition depends on the quality and diversity of the dataset. Limited or imbalanced image data

for certain gestures can reduce model performance. Variations in hand shape, orientation, lighting, and background also affect prediction accuracy.

3.5.2. Computation Constraints: The system is designed to run on standard hardware such as a laptop with a webcam. Therefore, model complexity, image size, and processing speed must be optimized to ensure smooth real-time performance without requiring high-end GPUs.

3.5.3. Architectural Constraints: The system follows an offline training approach where the model is trained beforehand and then used for real-time prediction. Continuous learning or real-time model updates are not included in the current implementation.

3.5.4. Real-Time Processing Constraints: The system must process video frames quickly to provide instant feedback. This requires efficient algorithms for hand detection, preprocessing, and prediction to avoid delays or lag during execution.

3.5.5. Gesture Limitations: The system is limited to a predefined set of gestures used during training. It cannot recognize new or unseen gestures beyond the trained dataset.

3.5.6. Evaluation Constraints: The system is tested in a controlled environment using a limited set of gestures. Therefore, its performance in diverse real-world scenarios may vary and requires further testing for generalization.

CHAPTER -4: PROJECT REQUIREMENTS / METHODOLOGY

4. Project Requirements / Methodology:

4.1. Hardware Requirements:

Table 4.1: Hardware Requirements

Category	Specification
CPU (Minimum)	Dual-core 64-bit processor
RAM (Minimum)	8 GB
Storage (Minimum)	10+ GB free (raw .json.gz, processed CSVs, artifacts, build files)
Network	Internet required for initial setup
CPU (Recommended)	5+ cores (Intel i5 / Ryzen 5 or better)
RAM (Recommended)	8 GB
Storage (Recommended)	SSD with 20+ GB free
Optional	Docker-capable system (docker-compose with Redis)

4.2. Software Requirements:

Table 4.2: Software Requirements

Category	Specification
Core Stack (from project)	
OS	Linux / Windows / macOS (project is cross-platform)
Programming Language	Python 3.10+
Development Environment	VS Code / Jupyter Notebook
Libraries Used	OpenCV, NumPy, TensorFlow/Keras, cvzone, pyttsx3
Python	3.11 (offline pipeline; venv311 present)
Python Libraries used in training	
Scientific/Data	pandas, numpy, scipy
Image Processing	OpenCV
Numerical Computation	NumPy
Model Training	TensorFlow / Keras
Service/runtime dependencies	
Input Device	Webcam
Output	Display Screen + Audio (Speaker)

Data Handling	opencsv
Hand Tracking	cvzone
Text-to-Speech	pyttsx3
Optional Data/Caching	spring-boot-starter-data-redis (optional Redis caching via docker-compose.yml)

4.3. Dataset Description:

Table 4.3: Dataset Description

Category	Specification
Data Source	Self-collected dataset using webcam
Raw files used	Image-based hand gesture dataset
Classes/Gesture	Hello, Yes, No, Please, Love You, etc.
Data Collection Method	Real-time image capture using webcam

4.4. Mathematical Model (Gesture classification): The sign language detection problem is modeled as an image classification task. Let the dataset consist of input images and corresponding gesture labels:

Let:
$X = \{x_1, x_2, ..., x_n\}$ be the set of input images
$Y = \{y_1, y_2, ..., y_n\}$ be the set of corresponding gesture labels

The model learns a mapping function:

y = f(x)

Where:

x represents the input image (hand gesture)

y represents the predicted gesture label

f represents the trained machine learning model

4.4.1. User Similarity: The system can compute similarity between users based on their interaction patterns and recommend items preferred by similar users.

4.4.2. Item Similarity: The system can compute similarity between items based on user interactions and recommend items similar to those already interacted with by the user.

4.4.3. Latent Factor Model: The system can learn low-dimensional latent representations of users and items, where both are mapped into a shared feature space to capture hidden relationships and preferences.

4.5. Cosine Similarity: The project uses cosine similarity as the main similarity function because it measures directional closeness between vectors and is robust for sparse recommendation data.

$$sim(a, b) = (a \cdot b) / (||a|| \, ||b||) \tag{4.5}$$

Where:

$a \cdot b$ is the dot product.

$|| a ||$ and $|| b ||$ are the vector norms.

If either vector has zero magnitude, similarity is defined as 0 to avoid division by zero and to preserve stable predictions.

4.6. Prediction Principal: For a target user *u* and item *i*, the recommendation score is computed from the collaborative signal and then used to rank items in descending order. The general prediction objective is:

$$\hat{y}ui = f(u, i) \tag{4.6}$$

where *f* is a similarity-based or factor-based scoring function.

4.7. Latent Factor Interpretation: The system also represents users and items as dense vectors learned during offline training:

p_u = collaborative latent vector for user *u*

q_i = collaborative latent vector for item *i*

c_u = content profile vector for user *u*

d_i = content feature vector for item *i*

The collaborative score and content score are computed as:

$$s_{cf} = sim(p_u, q_i) \tag{4.7}$$

$$s_{content} = sim(c_u, d_i) \tag{4.7}$$

The final hybrid score is:

$$score(u, i) = w \cdot cos(x_u, y_i) + (1 - w) \cdot cos(c_u, c_i) \tag{4.7}$$

Where, α is the collaborative weight. In the project implementation, this weight is controlled by configuration parameters such as *wcf* and *wContent*.

4.8. Algorithms Used: The project uses computer vision and machine learning-based algorithms to perform real-time hand gesture recognition and convert it into text and speech.

4.8.1.1. Hand Detection Algorithm: Capture live video frames using webcam.

4.8.1.2. Use HandDetector (cvzone) to detect hand landmarks.

4.8.1.3. Extract bounding box around the detected hand

4.8.1.4. Predict the score of an unseen item from the neighbors Interactions.

Select the primary hand region for further processing

The similarity between two users is:

$$\mathrm{sim}(u, v) = \frac{R_u \cdot R_v}{\|R_u\| \, \|R_v\|} \qquad (4.8.1)$$

Where: $N_k(u)$ and R_v are the user interaction vectors.

The predicted preference for user u on item i can be written as:

ŷ = argmax P(y|x)

Where:

ŷ = predicted gesture

P(y|x) = probability of gesture given input image

Where:

$\bar{r}_u$ is the average behavior of user u,

r_{vi} is the neighbor v's 0 with item i,

The denominator normalizes the weighted contribution of neighbors.

4.8.2. Item-Item Collaborative Filtering: Item-item collaborative filtering recommends items similar to those a user has already interacted with. It is often more stable than user-based filtering because item behavior changes more slowly than user behavior.

Working Steps:

4.8.2.1. Construct item vectors from user interaction patterns.

4.8.2.2. Compute similarity between items i and j.

4.8.2.3. For a given user, examine items they liked or interacted with.

4.8.2.4. Aggregate scores from similar items.

4.8.2.5. Recommend the highest-scoring unseen items.

4.8.2.6. more scalable than user-user CF in many cases,

The item similarity is computed as:

Where,

$S(u)$ is the set of items already interacted with by user u.

4.8.3. Real-Time Gesture Recognition Strategy: The system uses a hybrid processing approach that combines computer vision techniques with machine learning-based classification to achieve accurate real-time sign language detection. This approach helps overcome the limitations of using only image processing or only machine learning independently.

Working Steps:

4.8.3.1. Detect hand using computer vision (bounding box detection)

4.8.3.2. Preprocess the image (resize, normalize, background adjustment)

4.8.3.3. Generate prediction using trained model (CNN classifier)

4.8.3.4. Convert predicted gesture into text output

Convert text into speech using text-to-speech engine

The system combines detection and classification as:

- **Detection Component:** Extracts hand region from image
- **Classification Component:** Identifies gesture using trained model

The final prediction is based on:

$$\hat{y} = \arg\max P(y \mid x)$$

Where:

x= input image

y= gesture class

$\hat{y}$= predicted gesture

Why this approach used:

4.8.3.5. Improves accuracy in real-time detection

4.8.3.6. Reduces noise and false predictions

4.8.3.7. Works efficiently on standard hardware

4.8.3.8. Handles variations in hand position and lighting

4.8.3.9. Provides both visual (text) and audio (speech) output.

CHAPTER -5: PROJECT DEVELOPMENT

5. Project Development:

5.1. Software Development Model: The project follows a **simple iterative and experimental development approach** rather than a strict Waterfall flow. Development progressed in multiple stages: (1) data collection using webcam, (2) image preprocessing, (3) model training using Teachable Machine, (4) real-time system implementation using OpenCV and cvzone, (5) testing and performance improvement, and (6) final integration of text and speech output.

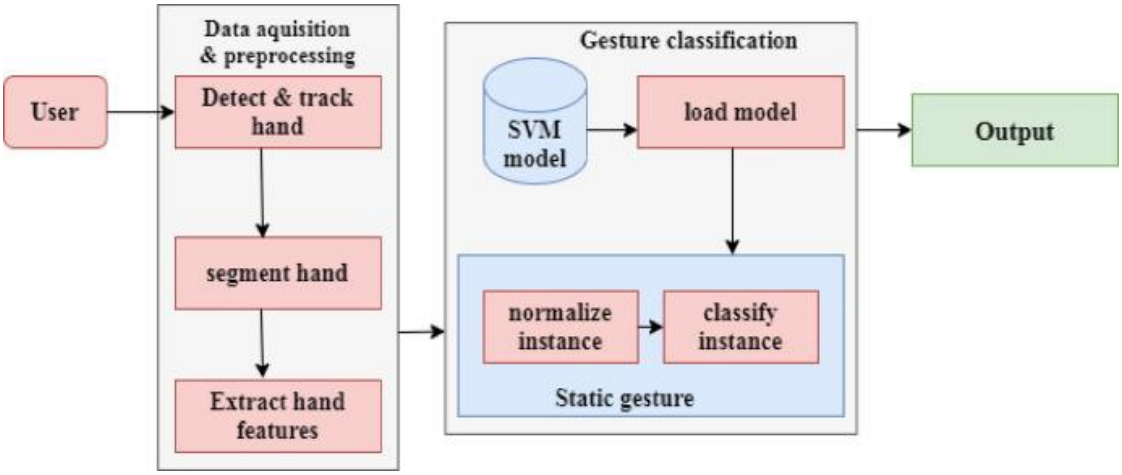


Figure 5.1: Simple iterative & experimental development approach

The diagram illustrates of the Simple iterative & experimental development approach and making more suitable for iterative model development in recommendation systems.

5.2. System Design (Real-Time Processing Architecture): The system is implemented using Python and follows a real-time processing pipeline instead of a traditional backend framework. Core responsibilities include capturing video input, detecting hand gestures, preprocessing images, performing gesture classification, and generating text and speech output.

Key system design elements:

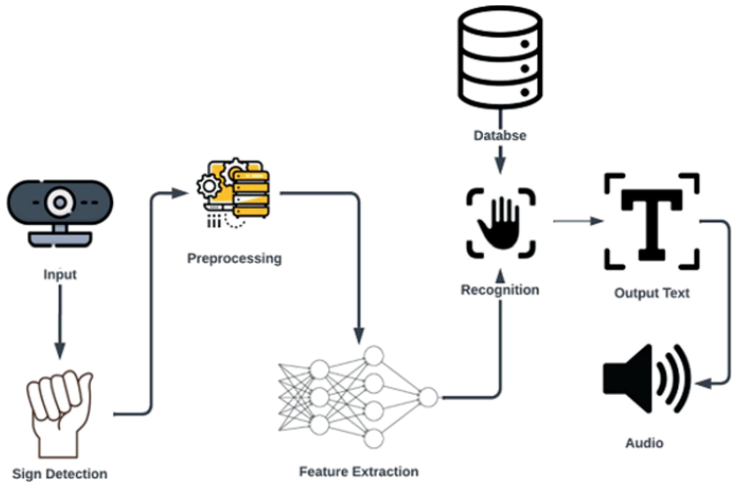


Figure 5.2: Layered Architecture of Real-Time Sign Language Detection System

The diagram illustrates the layered architecture of the system. The input layer captures real-time video, the processing layer performs hand detection and image preprocessing, the model layer handles gesture classification, and the output layer generates text and speech. An optimization layer ensures smooth performance and avoids repeated predictions.

5.2.1. Input Layer: Handles real-time video capture using webcam and provides continuous frames for processing.

5.2.2. Processing Layer: Performs: Hand detection using cvzone HandDetector, Extraction of hand region (bounding box), Image preprocessing (resize, normalization, background adjustment)

5.2.3. Model Layer: Loads trained model (keras_model.h5), Performs gesture classification, Outputs predicted label with confidence score.

5.2.4. Output Layer: Displays detected gesture as text, Converts text to speech using pyttsx3, Plays audio output through speakers.

5.2.5. Optimization Layer: Applies confidence threshold, Reduces repeated voice output, Improves real-time performance.

The system is designed to work efficiently in real-time, with all processing done locally without requiring external servers or APIs. This ensures fast response and easy deployment.

5.3. User Interface Design: The system does not include a traditional frontend such as a web or mobile application. Instead, it uses a **real-time visual interface** generated through OpenCV.

The interface directly displays:

- Live webcam feed
- Bounding box around detected hand
- Predicted gesture label
- Processed image window

5.3.1. System Workflow:

5.3.1.1. Input Capture: Hand Detection

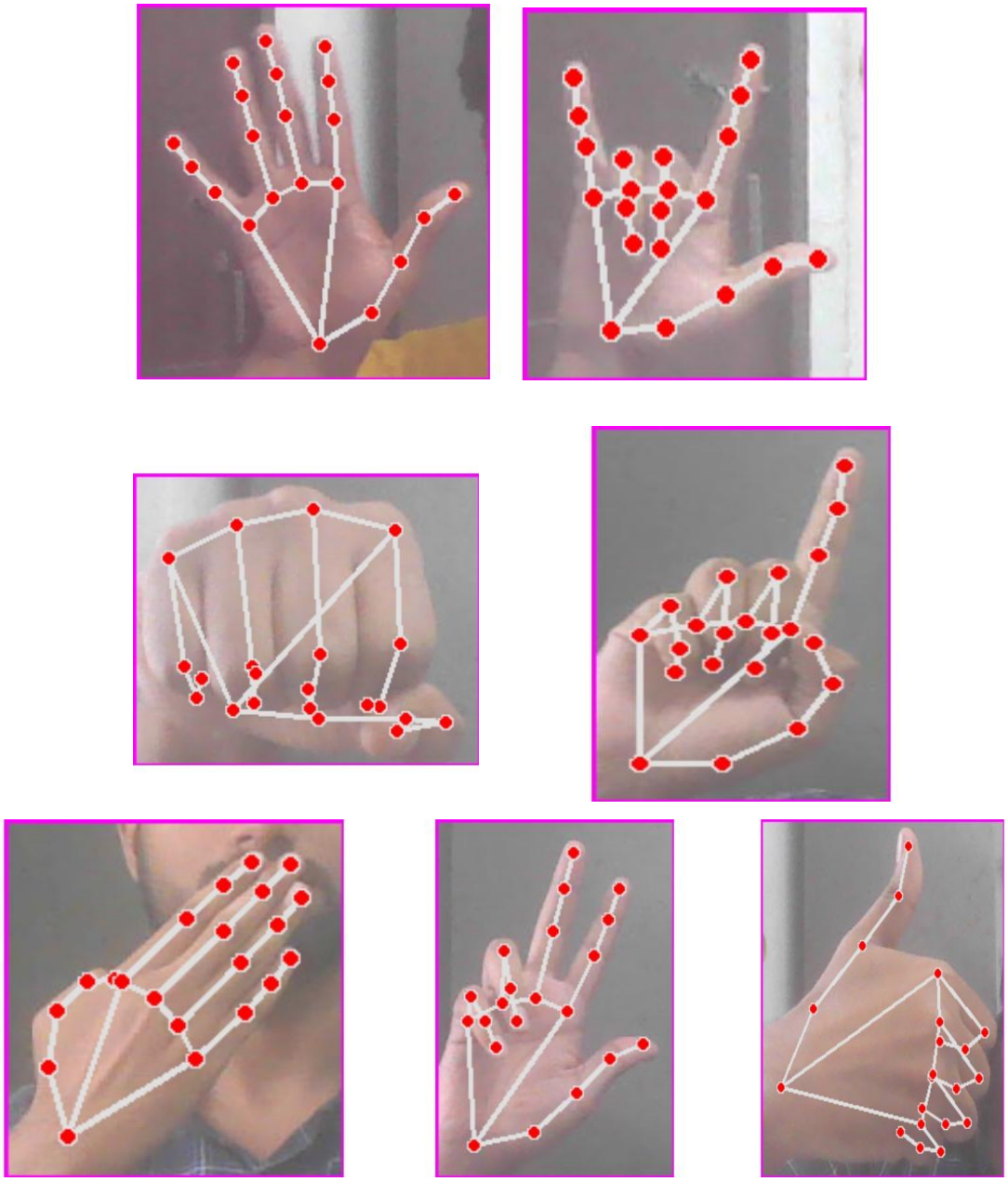


Figure 5.3.1.1: System Workflow 1

5.3.1.2. Image Preprocessing:

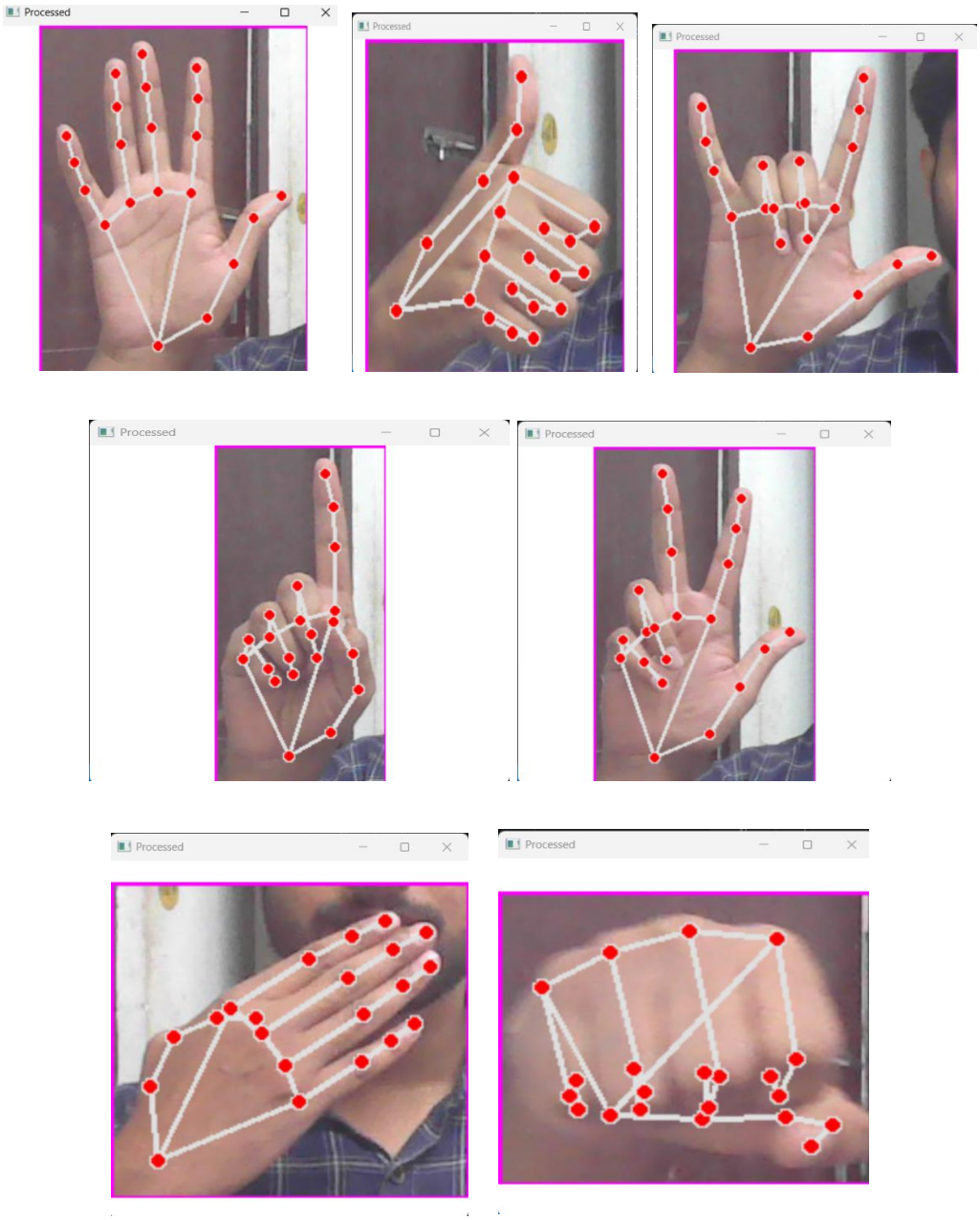
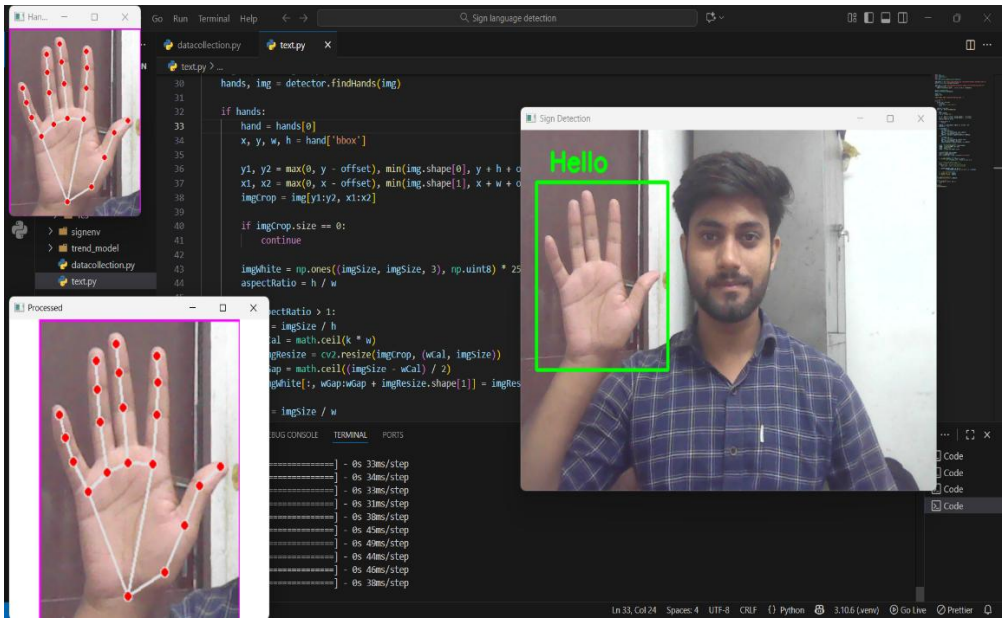
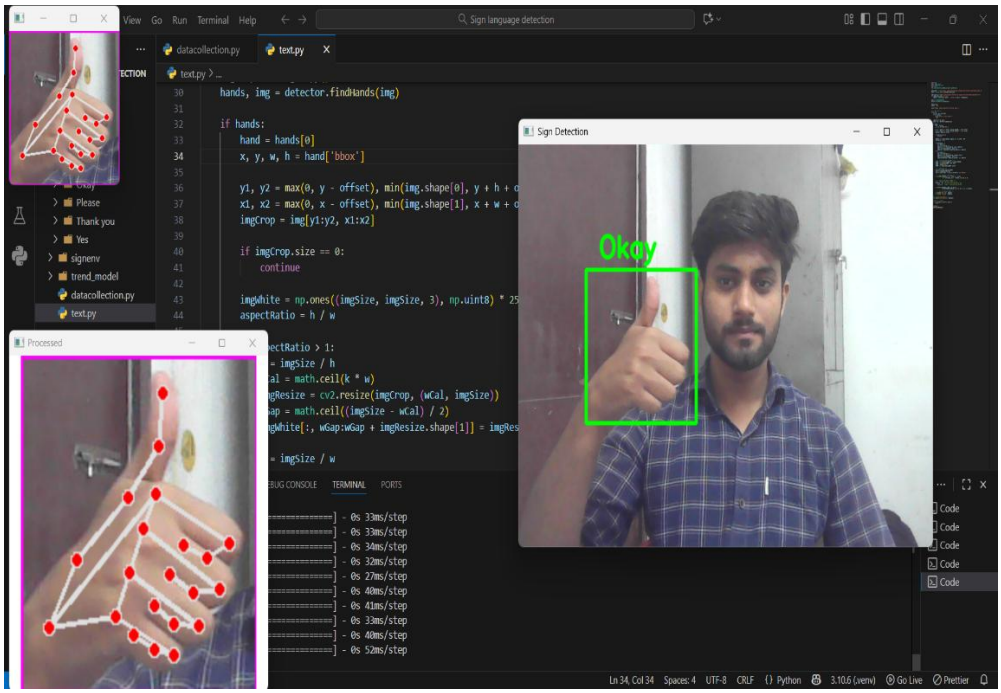
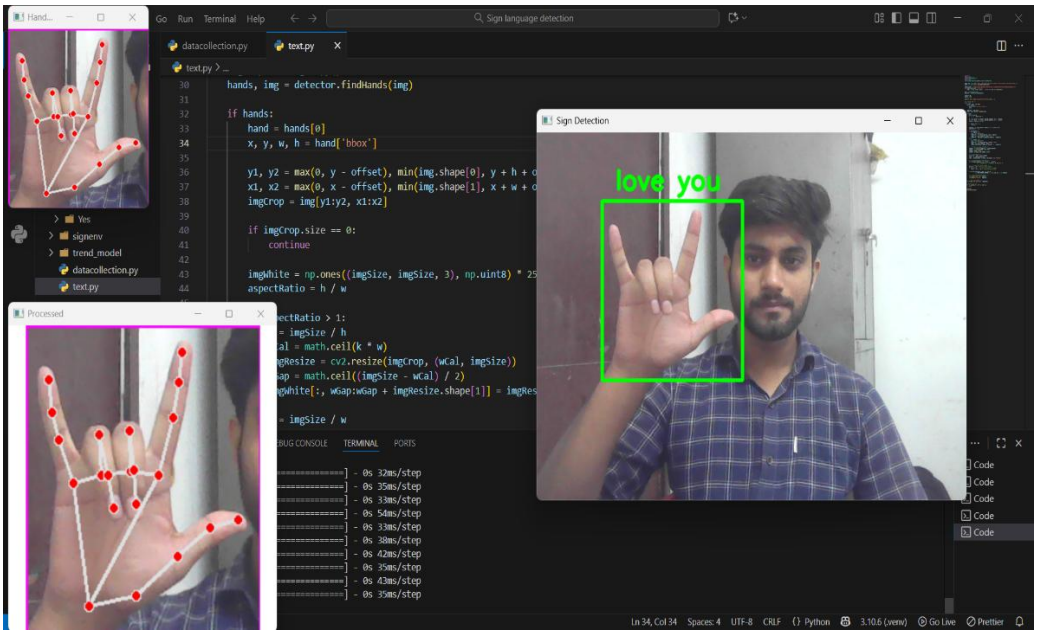


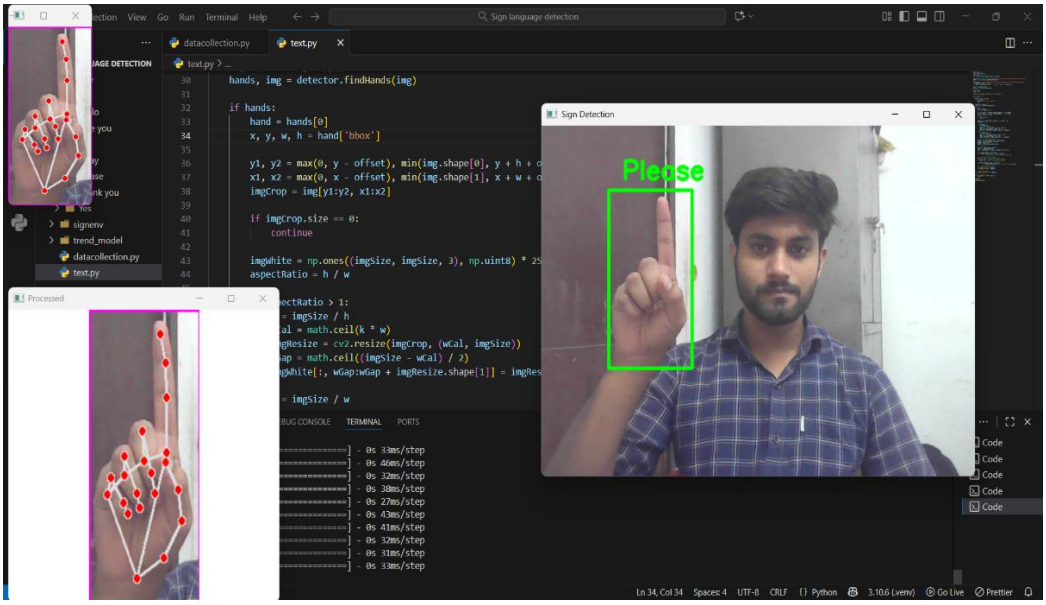
Figure 5.3.1.2. System workflow 2

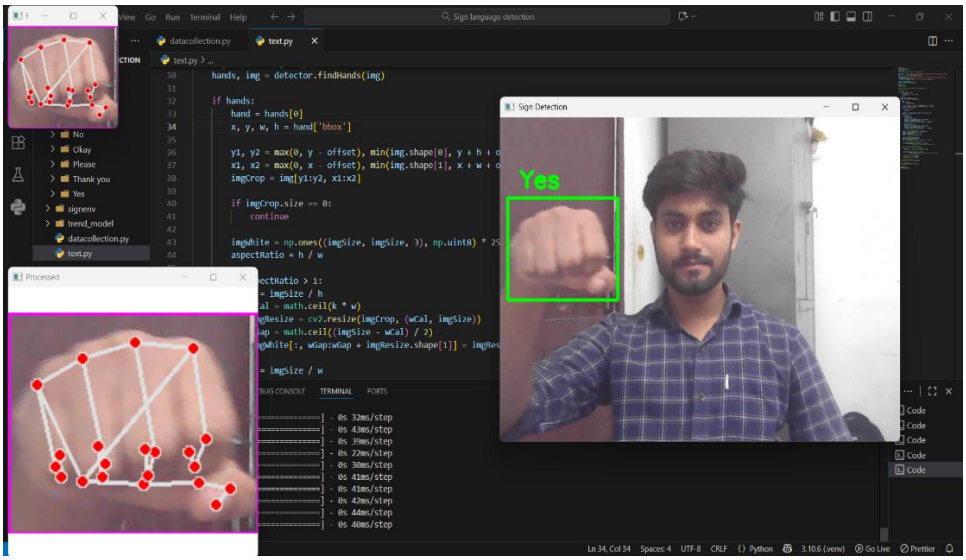
5.3.2. Gesture Prediction: Output Generation






Sign Detection
love you
Processed
datacollection.py
text.py
hands, img = detector.findHands(img)
if hands:
hand = hands[0]
x, y, w, h = hand['bbox']
if imgCrop.size == 0:
continue
TERMINAL
PORTS


Sign Detection
Please
Processed
datacollection.py
text.py
hands, img = detector.findHands(img)
if hands:
hand = hands[0]
x, y, w, h = hand['bbox']
if imgCrop.size == 0:
continue
TERMINAL
PORTS



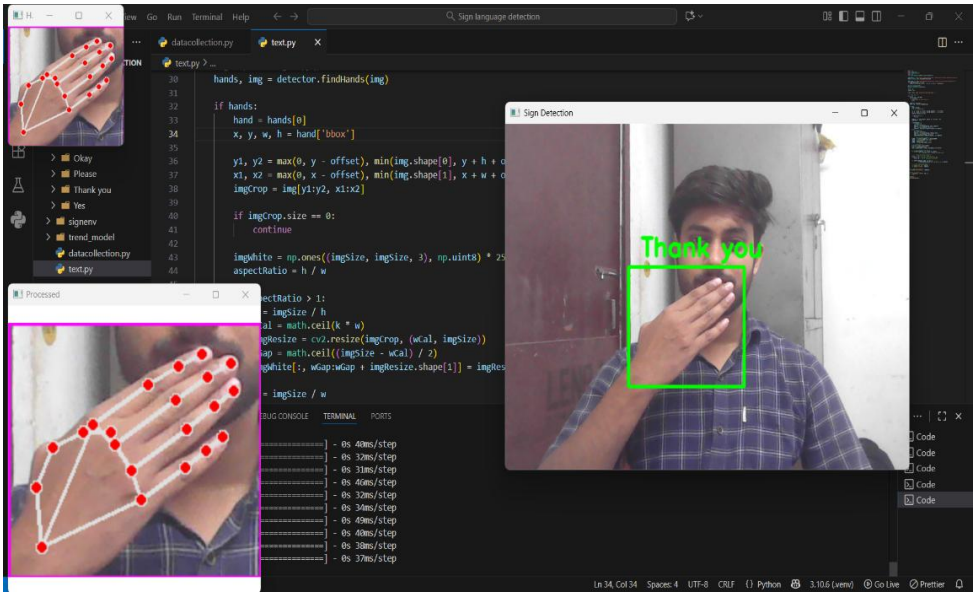


Figure 5.3.2: Gesture Output workflow

5.4. Coding Implementation

Implementation is split into offline and online components:

5.4.1. Offline Data & Training (Python): The offline phase focuses on data collection and model training. Hand gesture images are captured using a webcam and stored in labeled folders for each sign (e.g., Hello, Yes, No, Love You).

5.4.1.1. Data preprocessing Captured images are processed to maintain uniformity:

Code Snippet 5.4.1.1: Libraries Import

```
import tensorflow as tf
import cv2
import mediapipe as mp
import cvzone
import numpy as np

print("TensorFlow:", tf.__version__)
print("NumPy:", np.__version__)
print("OpenCV:", cv2.__version__)
print("Mediapipe:", mp.__version__)
print("CVZone:", cvzone.__version__)
```

5.4.1.2. Data Collection & preparation: The detected hand region is:

- Cropped using bounding box
- Resized to model input size (224×224)
- Converted to normalized format
- Cropping hand region
- Resizing images to fixed dimensions
- Normalizing pixel values

Code Snippet 5.4.1.2: Collaborative Filtering ana preparation

```
import cv2
from cvzone.HandTrackingModule import HandDetector
import numpy as np
import math
import os

cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)

offset = 20
imgSize = 300


labels = ["Hello", "Yes", "No", "ThankYou", "Please", "LoveYou", "Help"]
current_label = labels[0]


base_folder = "Data"


for label in labels:
    path = os.path.join(base_folder, label)
    os.makedirs(path, exist_ok=True)

counter = 0

while True:
    success, img = cap.read()
    if not success:
        continue

    hands, img = detector.findHands(img)
```

```
if hands:
    hand = hands[0]
    x, y, w, h = hand['bbox']

    imgWhite = np.ones((imgSize, imgSize, 3), np.uint8) * 255

    imgCrop = img[y-offset:y+h+offset, x-offset:x+w+offset]

    if imgCrop.size == 0:
        continue

    aspectRatio = h / w

    if aspectRatio > 1:
        k = imgSize / h
        wCal = math.ceil(k * w)
        imgResize = cv2.resize(imgCrop, (wCal, imgSize))
        wGap = (imgSize - wCal) // 2
        imgWhite[:, wGap:wGap+wCal] = imgResize
    else:
        k = imgSize / w
        hCal = math.ceil(k * h)
        imgResize = cv2.resize(imgCrop, (imgSize, hCal))
        hGap = (imgSize - hCal) // 2
        imgWhite[hGap:hGap+hCal, :] = imgResize

    cv2.imshow("ImageWhite", imgWhite)

cv2.putText(img, f"Label: {current_label}", (10, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

```
        cv2.imshow("ImageWhite", imgWhite)

    cv2.putText(img,  (constant) FONT_HERSHEY_SIMPLEX: int
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    cv2.imshow("Image", img)

    key = cv2.waitKey(1)


    if key == ord('s'):
        folder = os.path.join(base_folder, current_label)
        cv2.imwrite(f'{folder}/Image_{counter}.jpg', imgWhite)
        counter += 1
        print(f"{current_label} Images:", counter)


    elif key == ord('n'):
        idx = labels.index(current_label)
        current_label = labels[(idx + 1) % len(labels)]
        counter = 0
        print("Switched to:", current_label)

    # Quit
    elif key == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

5.4.1.3. Model Training:

The processed dataset is used to train a gesture classification model using Teachable Machine, which generates a TensorFlow/Keras model file (keras_model.h5) and a label file (labels.txt).

5.4.1.4. Model Output:

The trained model learns patterns of hand gestures and is exported for real-time use.

Code Snippet 5.4.1.3: labels.txt

trend_model > labels.txt

```
Hello
love you
No
Okay
Please
Thank you
Yes
Help
Excuse me
Come
Go
```

Code snippet 5.4.1.4 Text.py

```
import cv2
import pandas as pd,
import numpy as np,
from scipy import sparse
import tensorflow as tf
from sklearn.preprocessing import normalize
import math
from cvzone.HandTrackingModule import HandDetector
import pyttsx3
import time
```

Configuration

```
import cv2
import numpy as np
import tensorflow as tf
import math
from cvzone.HandTrackingModule import HandDetector
import pyttsx3
import time


model_path = r"C:\Users\Sahil Siddique\Desktop\Sign language detection\trend_model\keras_model.h5"
model = tf.keras.models.load_model(model_path)

labels_path = r"C:\Users\Sahil Siddique\Desktop\Sign language detection\trend_model\labels.txt"
with open(labels_path, "r") as f:
    labels = [line.strip().split(' ', 1)[-1] for line in f.readlines()]


cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)

offset = 20
imgSize = 400


engine = pyttsx3.init()
engine.setProperty('rate', 170)
engine.setProperty('volume', 1.0)

voices = engine.getProperty('voices')
engine.setProperty('voice', voices[0].id)


prev_label = ""
last_spoken_time = 0
```

```
sentence = []

print("Model loaded successfully! Starting camera...")

while True:
    success, img = cap.read()
    if not success:
        break

    imgOutput = img.copy()
    hands, img = detector.findHands(img)

    if hands:
        hand = hands[0]
        x, y, w, h = hand['bbox']

        y1, y2 = max(0, y - offset), min(img.shape[0], y + h + offset)
        x1, x2 = max(0, x - offset), min(img.shape[1], x + w + offset)
        imgCrop = img[y1:y2, x1:x2]

        if imgCrop.size == 0:
            continue

        imgWhite = np.ones((imgSize, imgSize, 3), np.uint8) * 255
        aspectRatio = h / w

        if aspectRatio > 1:
            k = imgSize / h
            wCal = math.ceil(k * w)
            imgResize = cv2.resize(imgCrop, (wCal, imgSize))
            wGap = (imgSize - wCal) // 2
            imgWhite[:, wGap:wGap + wCal] = imgResize
        else:
            k = imgSize / w
            hCal = math.ceil(k * h)
            imgResize = cv2.resize(imgCrop, (imgSize, hCal))
```

```
    hCal = math.ceil(k * h)
    imgResize = cv2.resize(imgCrop, (imgSize, hCal))
    hGap = (imgSize - hCal) // 2
    imgWhite[hGap:hGap + hCal, :] = imgResize


imgRGB = cv2.cvtColor(imgWhite, cv2.COLOR_BGR2RGB)
imgRGB = cv2.resize(imgRGB, (224, 224))
imgRGB = imgRGB / 255.0
imgRGB = np.expand_dims(imgRGB, axis=0)


prediction = model.predict(imgRGB, verbose=0)
index = np.argmax(prediction)
confidence = prediction[0][index]

label = labels[index] if index < len(labels) else "Unknown"

if confidence > 0.75:
    current_time = time.time()


    if label != prev_label or (current_time - last_spoken_time) > 2:
        engine.say(label)
        engine.runAndWait()

        sentence.append(label)
        prev_label = label
        last_spoken_time = current_time


cv2.rectangle(imgOutput, (x - offset, y - offset),
              (x + w + offset, y + h + offset), (0, 255, 0), 3)

text = f"{label} ({round(confidence*100,1)}%)"
text_y = y - 35 if y - 35 > 35 else y + h + 45
```

```
        cv2.rectangle(imgOutput, (x - offset, y - offset),
                      (x + w + offset, y + h + offset), (0, 255, 0), 3)

        text = f"{label} ({round(confidence*100,1)}%)"
        text_y = y - 35 if y - 35 > 35 else y + h + 45

        cv2.putText(imgOutput, text, (x, text_y),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        cv2.imshow("Processed", imgWhite)


    cv2.putText(imgOutput, "Sentence: " + " ".join(sentence[-6:]),
                (10, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 0), 2)

    cv2.imshow("Sign Detection", imgOutput)

    key = cv2.waitKey(1)

    if key == ord('c'):
        sentence = []

    if key == ord('v'):
        full_sentence = " ".join(sentence)
        engine.say(full_sentence)
        engine.runAndWait()

    if key == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

5.5. Image Data collection Mechanism : The following module is responsible for collecting and preparing the dataset for training the sign language detection model. It captures hand gesture images using a webcam, processes them, and stores them in organized folders based on gesture labels.

5.6. System Initialization: The system begins by importing the required libraries such as OpenCV for image processing, NumPy for numerical operations, and cvzone's HandDetector for detecting hands in real time.

The webcam is initialized using:

```
cap = cv2.VideoCapture(0)
```

This module is used to collect and prepare the dataset required for training the real-time sign language detection system. The system captures hand gesture images using a webcam and stores them in an organized manner according to different gesture labels. This dataset plays an important role in improving the accuracy of the trained model.

Initially, the webcam is activated using OpenCV, which continuously captures live video frames. Each frame is processed in real time to detect the presence of a hand. For this purpose, a hand detection module from cvzone is used, which identifies the hand and provides its position in the form of bounding box coordinates. This allows the system to focus only on the hand region instead of the entire frame.

Once the hand is detected, the system extracts the region of interest (ROI) by cropping the hand portion from the image. A small offset is added around the detected hand so that the complete gesture is captured properly without cutting any part. This helps in improving the quality of the dataset.

After cropping, the image is further processed to maintain consistency across all samples. A white background image of fixed size is created, and the cropped hand image is resized and placed inside it. During this process, the aspect ratio of the hand is preserved so that the image does not get distorted. This ensures

that all images have uniform size and structure, which is necessary for effective model training.

The system also displays both the original camera feed and the processed image in real time. This helps the user to monitor whether the hand is properly detected and processed. Additionally, the current gesture label is displayed on the screen so that the user knows which category of images is being collected at that moment.

5.7. Model Training and Real-Time Detection Mechanism: After the successful collection of image data, the next important phase of the system is model training and real-time gesture detection. This phase is responsible for enabling the system to recognize different hand gestures and convert them into meaningful output in the form of text and speech.

In the model training stage, the collected dataset is used to train a gesture classification model. The dataset contains multiple images of different hand gestures, each stored in separate folders according to their respective labels such as Hello, Yes, No, Thank You, and others. These labeled images help the model learn patterns and differences between various gestures.

For training the model, Teachable Machine is used, which is a user-friendly tool that allows quick training of image classification models. The collected images are uploaded into the tool, where each folder represents a separate class. The model is then trained using these images, and it learns to distinguish between different hand gestures based on visual features such as shape, orientation, and position of the hand.

After the training process is completed, the model is exported in TensorFlow/Keras format as a file named keras_model.h5, along with a label file (labels.txt). These files are essential for performing real-time predictions in the system. The trained model contains the learned patterns, while the label file maps prediction outputs to their respective gesture names.

Once the model is ready, it is integrated into the real-time detection system. The system again uses the webcam to capture live video frames. Each frame is processed in real time, where the hand is detected using the same hand detection technique used during data collection. The region of interest is cropped and preprocessed in the same way to ensure consistency between training data and real-time input.

The preprocessed image is then converted into the required format for the model. This includes resizing the image to a fixed size (such as 224×224 pixels), normalizing pixel values, and converting it into a numerical array. The processed image is then passed to the trained model for prediction.

The model generates a probability score for each gesture class, and the class with the highest probability is selected as the final prediction. This predicted label is then displayed on the screen along with the live video feed. A bounding box is drawn around the detected hand to visually indicate the region being analyzed.

To improve the accuracy and reliability of the system, a confidence threshold is applied. Only predictions with a confidence level above a certain value are considered valid. This helps in reducing incorrect predictions and improves overall system performance.

In addition to text output, the system also converts the detected gesture into speech using a text-to-speech engine. This allows the system to provide audio feedback, making it more interactive and useful for communication purposes. The voice output is generated only when a stable prediction is detected, which helps in avoiding repeated or unnecessary audio output.

The system continuously repeats this process for every frame captured by the webcam, enabling real-time gesture recognition. This ensures smooth and fast interaction between the user and the system. The combination of image

processing, machine learning, and real-time execution makes the system efficient and practical for real-world use.

Overall, this phase completes the core functionality of the project. It transforms raw image data into a functional intelligent system capable of recognizing sign language gestures and converting them into understandable text and speech output. This integration of training and real-time detection is the key component that makes the system effective and reliable.

5.7.1. Primary Integrated Endpoints:

5.7.2. Table 5.5.1: Primary Integrated Endpoints:

Table 5.5.1: API Endpoints Summary

Function	Input	Description
Camera Initialization	System Start	Activates webcam for real-time video capture
Hand Detection	Video Frame	Detects hand and generates bounding box using cvzone
Image Preprocessing	Cropped Image	Cropped Image
Gesture Prediction	Processe d Image	Predicts gesture using trained model (keras_model.h5)
Text Output	Predicted Label	Displays detected gesture on screen
Voice Output	Text	Converts text into speech using pyttsx3
Image Capture	Key Press ('s')	Key Press ('s')
Label Switching	Key Press ('n')	Switches between gesture categories

This endpoint set supports both prototype real time integration and independent camera testing, while preserving clear separation between training data.

5.7.3. Integration Flow: The interaction between system components follows a structured real-time processing pipeline. The system continuously captures video input from the webcam, which serves as the primary input source. Each frame is then passed to the hand detection module, where the hand region is identified using computer vision techniques.

The detected hand region is forwarded to the preprocessing module, where the image is cropped, resized, and normalized to match the input requirements of the trained model. This ensures consistency between the training data and real-time input.

The processed image is then passed to the gesture classification model, where prediction is performed using the trained TensorFlow/Keras model. The model generates probability scores for each gesture class, and the gesture with the highest confidence is selected as the final output.

The predicted gesture is then sent to the output module, where it is displayed as text on the screen. Additionally, the text is converted into speech using a text-to-speech engine, providing audio feedback to the user.

The system continuously repeats this process for each frame, enabling real-time detection and interaction. This pipeline ensures smooth data flow between components and efficient execution without the need for external servers or request-response mechanisms.

At the initial stage, the system captures live video input using a webcam. The video stream is divided into individual frames, which are processed sequentially. Each frame acts as an independent input to the system. This continuous stream of frames allows the system to monitor hand movements in real time and respond instantly.

Once a frame is captured, it is passed to the hand detection module. This module uses computer vision techniques through the cvzone HandDetector to identify the presence of a hand in the frame. If a hand is detected, the module extracts the bounding box coordinates, which define the exact location of the hand. This step is essential because it isolates the relevant region from the entire image, reducing noise and improving processing efficiency.

After detecting the hand, the system forwards the extracted region to the preprocessing module. In this stage, the hand image is cropped using the bounding box coordinates with a small offset to ensure that the complete gesture is captured. The cropped image is then resized to a fixed dimension and placed on a white background. During this process, the aspect ratio is preserved to avoid any distortion in the image. The preprocessing step ensures that all input images are standardized, which is crucial for accurate model prediction.

The preprocessed image is then converted into the format required by the trained machine learning model. This includes resizing the image to the model's input size (typically 224×224 pixels), normalizing pixel values, and converting it into a numerical array. The processed data is then ready to be passed to the classification model.

The gesture classification module plays a central role in the system. It uses a trained TensorFlow/Keras model that has learned patterns from previously collected gesture images. When the processed image is fed into the model, it generates prediction probabilities for all available gesture classes. The system then selects the gesture with the highest probability as the final prediction. This approach ensures that the most likely gesture is identified accurately.

To enhance reliability, a confidence threshold is applied to the prediction results. If the confidence score of the predicted gesture is below a certain

level, the system may ignore the prediction to avoid incorrect outputs. This helps in improving the overall accuracy and reduces false detections during real-time execution.

Once a valid prediction is obtained, the result is passed to the output module. In this stage, the predicted gesture is displayed as text on the screen along with the live video feed. A bounding box is also drawn around the detected hand to visually indicate the area being analyzed. This visual feedback helps the user understand how the system is interpreting the gesture.

In addition to text output, the system also includes an audio output feature. The predicted gesture is converted into speech using a text-to-speech engine. This allows the system to provide voice feedback, making it more interactive and useful, especially for communication purposes. To avoid repeated or unnecessary voice output, a delay or control mechanism is applied so that the same gesture is not spoken continuously.

The entire process runs in a continuous loop, where each frame undergoes the same sequence of operations. This loop-based architecture ensures that the system responds instantly to changes in hand gestures, providing a seamless real-time experience. The integration of all modules—input capture, hand detection, preprocessing, model prediction, and output generation—creates a cohesive and efficient system.

Overall, the integration flow of the system ensures smooth communication between all components without the need for complex backend infrastructure. Each module performs a specific function and passes its output to the next stage, forming a well-organized pipeline. This design not only improves performance but also makes the system scalable and easy to understand. The real-time nature of the pipeline is the key factor that enables effective sign language recognition and interaction.

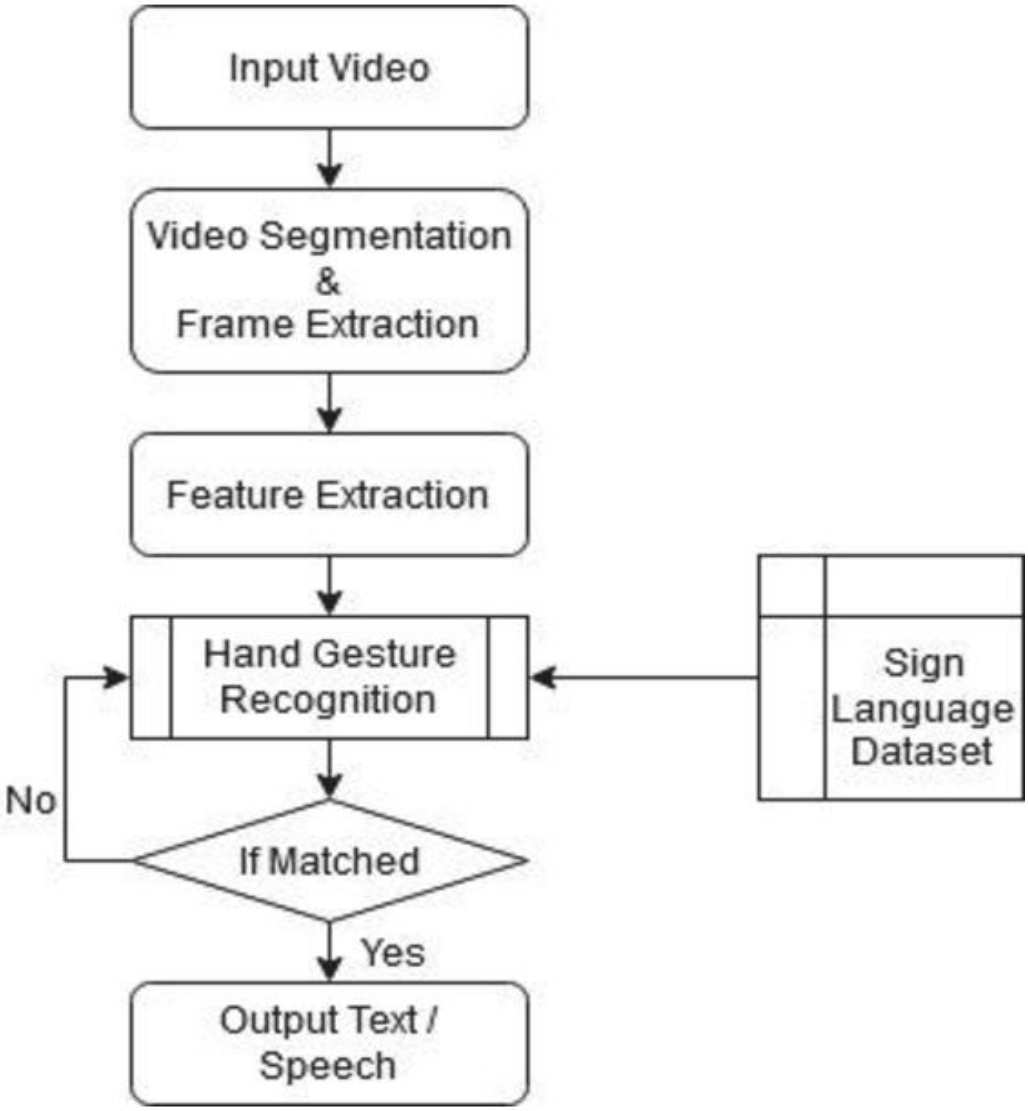


Figure 5.5.2: Integration Flow Diagram *1*

CHAPTER -6: TESTING

6. Testing:

6.1. Unit Testing: Unit testing in this project focuses on verifying the correctness of individual components such as image preprocessing, hand detection, and model prediction. Since the system is implemented in Python and operates in real time, testing is primarily performed by validating each module independently during development.

The system consists of multiple independent components such as video capture, hand detection, image preprocessing, model prediction, and output generation. Each of these components is tested individually to ensure that they function as expected before being integrated into the complete pipeline.

The primary unit test scope includes image preprocessing operations such as cropping, resizing, and normalization. These steps are crucial because the trained model requires input images of a fixed size and format. Therefore, it is tested whether the cropped hand image is correctly centered, resized to the required dimensions, and placed on a white background without distortion. Special attention is given to maintaining the aspect ratio of the image so that gesture features are preserved accurately.

Another important component tested is the hand detection module. The cvzone HandDetector is used to detect the hand and generate bounding box coordinates. Unit testing ensures that the bounding box correctly encloses the hand region and adapts properly to different hand positions and orientations. The system is also tested to handle cases where no hand is detected, ensuring that it does not crash or produce invalid outputs.

Model prediction is also a critical part of unit testing. The trained TensorFlow/Keras model is tested using sample images from the dataset to verify whether it correctly identifies known gestures. The mapping between

prediction output and gesture labels is validated to ensure that the correct label is displayed for each detected gesture.

The text-to-speech module is also tested independently. It is verified that the system converts predicted text into audible speech correctly and that the speech output is clear and understandable. Additional checks are performed to ensure that the system does not repeatedly generate voice output for the same gesture without any change.

Primary unit test scope includes:

6.1.1. Image preprocessing operations such as cropping, resizing, and background normalization

6.1.2. Hand detection accuracy using bounding box coordinates.

6.1.3. Model prediction correctness using sample input images.

6.1.4. Label mapping from prediction output to actual gesture names.

6.1.5. Text-to-speech functionality for generating voice output

Representative verified scenarios:

6.1.6. Detected hand region is correctly cropped and centered in the processed image.

6.1.7. Preprocessed image maintains correct size and aspect ratio.

6.1.8. Model returns correct label for known gesture inputs.

6.1.9. Invalid or empty frames are safely ignored without crashing the system.

6.1.10. Voice output is triggered only for valid and stable predictions

6.1.11. Preprocessed image maintains correct size and aspect ratio

6.2. Integration Testing: Integration testing is performed to evaluate how different components of the system work together in a real-time environment. Since the system follows a continuous processing pipeline, integration testing focuses on validating the complete workflow from input capture to output generation.

In this stage, all modules such as webcam input, hand detection, preprocessing, model prediction, and output display are combined and tested as a single system. The goal is to ensure smooth data flow between these components and verify that the system performs efficiently without delays or interruptions.

The testing begins with verifying the video capture process. The system is tested to ensure that it can continuously capture frames from the webcam without lag or frame drops. These frames are then passed to the hand detection module, and it is verified that the system consistently detects the hand in different lighting conditions and positions.

The integration between preprocessing and model prediction is also tested. It is ensured that the processed image is correctly passed to the model and that predictions are generated without errors. The output of the model is then validated to check whether the predicted gesture is displayed correctly on the screen.

Another important aspect of integration testing is synchronization between text and voice output. The system is tested to ensure that when a gesture is detected, the corresponding text is displayed and the voice output is generated at the appropriate time. A delay mechanism is also tested to prevent repeated audio output for the same gesture, which improves user experience.

The system is also evaluated for stability during long runtime execution. It is tested whether the system can run continuously without crashes, memory issues, or performance degradation. This is important because real-time systems must handle continuous input efficiently.

Primary unit test scope includes:

6.2.1. Continuous video capture and frame processing without lag

6.2.2. Smooth transition between hand detection and preprocessing stages

6.2.3. Proper data flow from preprocessing to model prediction

6.2.4. Correct display of predicted gesture on the screen

6.2.5. Synchronization between text output and voice output

Representative verified scenarios:

6.2.6. Real-time detection successfully identifies gestures and displays results continuously.

6.2.7. Multiple gestures are detected correctly when switching between different hand signs.

6.2.8. System handles absence of hand gracefully without errors.

6.2.9. Voice output does not repeat unnecessarily for the same gesture.

6.2.10. System exits properly when user presses the quit key

These tests ensure that all components of the system work together seamlessly and provide accurate and reliable real-time performance.

Major Testing Code Implementation:

The major testing implementation of the real-time sign language detection system is designed to evaluate the performance, accuracy, and reliability of the trained model in a real-time environment. This phase ensures that all system

components work correctly when integrated together and are capable of handling continuous input from the user.

The testing process begins with loading the trained machine learning model and its corresponding label file. The model, which is trained using a labeled dataset of hand gestures, contains learned patterns that allow it to classify different signs. The label file maps the numerical output of the model to meaningful gesture names. This step is essential because it prepares the system for real-time prediction.

Once the model is loaded, the system initializes the webcam to capture live video input. The video stream is processed frame by frame, where each frame acts as an independent input sample. This continuous flow of frames enables the system to operate in real time and respond instantly to user gestures.

The captured frames are passed to the hand detection module, where the system identifies the presence and position of the hand. This is achieved using computer vision techniques that generate bounding box coordinates around the detected hand. By isolating the hand region, the system reduces unnecessary background information and focuses only on the relevant portion of the image.

After detecting the hand, the system performs image preprocessing to convert the cropped hand image into a suitable format for the model. This includes resizing the image to a fixed dimension, normalizing pixel values, and maintaining the aspect ratio to prevent distortion. A white background is often used to standardize the input and improve model performance. This preprocessing step ensures consistency between training data and real-time input.

The processed image is then passed to the trained model for prediction. The model generates probability scores for each gesture class, and the class with the highest probability is selected as the predicted output. Along with the predicted

label, a confidence score is also calculated, which indicates how certain the model is about its prediction.

To improve the reliability of the system, a confidence threshold is applied. Only predictions with confidence above a certain level are considered valid. This helps in reducing incorrect detections and enhances the overall accuracy of the system. Predictions below the threshold are ignored to avoid misleading outputs.

The predicted gesture is then displayed on the screen as text, along with a bounding box around the detected hand. This provides visual feedback to the user and helps in understanding how the system is interpreting the gesture.

In addition to text output, the system also includes a text-to-speech module that converts the predicted gesture into voice output. This feature enhances usability, especially for communication purposes. To prevent repetitive audio output, a cooldown mechanism is implemented, which ensures that the same gesture is not spoken continuously within a short time interval.

The testing implementation also evaluates the system's performance under different conditions such as varying lighting, hand positions, and background environments. It ensures that the system remains stable and responsive during continuous operation. Error handling mechanisms are also tested to ensure that the system does not crash when no hand is detected or when invalid input is received.

Code Snippet 6.3: signLanguageTests.py

```
import cv2
import numpy as np
import tensorflow as tf
import time
from cvzone.HandTrackingModule import HandDetector
import pyttsx3

model = tf.keras.models.load_model("trend_model/keras_model.h5")

with open("trend_model/labels.txt", "r") as f:
    labels = [line.strip().split(' ')[-1] for line in f.readlines()

cap = cv2.VideoCapture(0)
detector = HandDetector(maxHands=1)

engine = pyttsx3.init()
engine.setProperty('rate', 160)

IMG_SIZE = 300
OFFSET = 20
CONF_THRESHOLD = 0.75

last_label = ""
last_time = 0
cooldown = 2

print("System Testing Started... Press Q to exit")

while True:
    ret, frame = cap.read()
    if not ret:
        continue
```

```
output = frame.copy()
hands, frame = detector.findHands(frame)

if hands:
    x, y, w, h = hands[0]['bbox']

    y1, y2 = max(0, y - OFFSET), min(frame.shape[0], y + h + OFFSET)
    x1, x2 = max(0, x - OFFSET), min(frame.shape[1], x + w + OFFSET)

    crop = frame[y1:y2, x1:x2]

    if crop.size != 0:
        white = np.ones((IMG_SIZE, IMG_SIZE, 3), np.uint8) * 255

        ratio = h / w

        if ratio > 1:
            scale = IMG_SIZE / h
            new_w = int(scale * w)
            resized = cv2.resize(crop, (new_w, IMG_SIZE))
            gap = (IMG_SIZE - new_w) // 2
            white[:, gap:gap+new_w] = resized
        else:
            scale = IMG_SIZE / w
            new_h = int(scale * h)
            resized = cv2.resize(crop, (IMG_SIZE, new_h))
            gap = (IMG_SIZE - new_h) // 2
            white[gap:gap+new_h, :] = resized

        img = cv2.resize(white, (224, 224))
        img = img / 255.0
        img = np.expand_dims(img, axis=0)

        preds = model.predict(img)
        idx = np.argmax(preds)
        confidence = preds[0][idx]
```

```
        detected_label = labels[idx] if idx < len(labels) else "Unknown"

        cv2.rectangle(output, (x, y), (x+w, y+h), (255, 0, 0), 2)
        cv2.putText(output, f"{detected_label} ({confidence:.2f})",
                    (x, y-10), cv2.FONT_HERSHEY_SIMPLEX,
                    0.9, (255, 0, 0), 2)

        current_time = time.time()
        if confidence > CONF_THRESHOLD:
            if detected_label != last_label or (current_time - last_time) > cooldown:
                engine.say(detected_label)
                engine.runAndWait()
                last_label = detected_label
                last_time = current_time

        cv2.imshow("Processed Input", white)

    cv2.imshow("Testing Window", output)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

CHAPTER -7: RESULT ANALYSIS

7. **Result Analysis:** This section evaluates the performance and effectiveness of the real-time sign language detection system from both prediction accuracy and system reliability perspectives. The analysis is based on practical testing, model behavior during real-time execution, and overall usability of the system.

   The evaluation begins with analyzing the accuracy of gesture recognition. The trained model is tested using real-time hand gestures, and its predictions are observed for correctness. It is found that the system performs well for gestures that are clearly defined and properly trained in the dataset. Gestures such as Hello, Yes, No, and Thank You are recognized with high accuracy under normal conditions. However, slight variations in hand orientation or incomplete gestures may sometimes reduce prediction accuracy.

   Another important aspect of analysis is the consistency of predictions. The system is tested for continuous input to ensure that it does not produce random or unstable outputs. It is observed that when a gesture is held steadily, the system consistently predicts the correct label. To improve stability, a confidence threshold is applied, which filters out low-confidence predictions and reduces incorrect outputs.

   The real-time performance of the system is also evaluated. The system processes video frames continuously and generates predictions with minimal delay. This ensures smooth interaction between the user and the system. The integration of hand detection, preprocessing, and model prediction works efficiently, allowing the system to operate in real time without noticeable lag.

   The usability of the system is analyzed by observing how easily a user can interact with it. The visual feedback in the form of bounding boxes and text labels helps users understand the detection process. Additionally, the voice output feature enhances user experience by converting detected gestures into speech, making the system more practical for communication purposes.

System reliability is also tested under different conditions such as varying lighting environments, different hand positions, and background variations. It is observed that the system performs best under proper lighting conditions and when the hand is clearly visible. In low lighting or cluttered backgrounds, detection accuracy may decrease slightly. However, the system remains stable and does not crash under such conditions.

Error handling is another important part of the analysis. The system is tested in scenarios where no hand is present in the frame. In such cases, the system continues running without generating false outputs, which indicates proper handling of edge cases. Similarly, the system exits safely when required, ensuring stability and robustness.

The Similar Items page displays products that are closely related to a selected item based on hybrid item-item similarity. The system analyzes both collaborative interaction patterns and content features to identify items with similar characteristics. This functionality enables users to discover alternative or complementary products, improving recommendation relevance and user experience.

When a user with no prior interaction history is queried, the system activates a fallback mechanism and returns popular items. This ensures that the system remains functional even in cold-start scenarios.

7.1. Graphs (Training and testing accuracy):

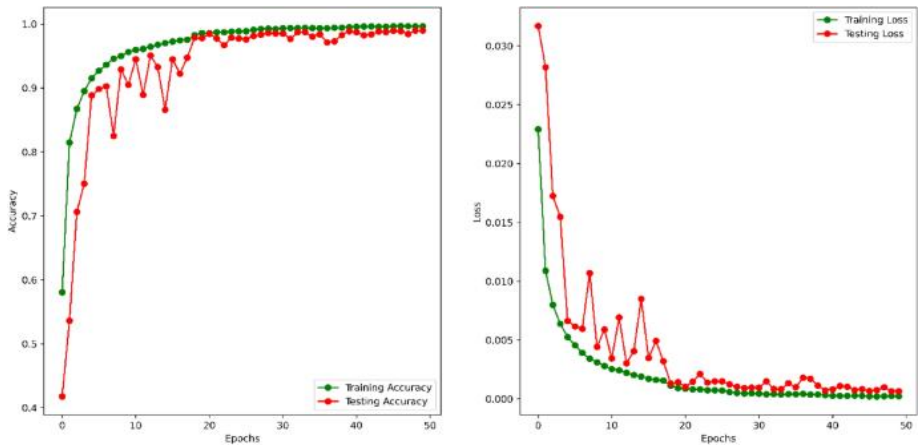


Figure 7.1: Figure 7.x: Performance Comparison of sign gesture Approaches

Precision: 89% - The proportion of true positive results compared to total positive predictions ● Recall: 85% - The model's ability to identify all relevant instances of particular emotions ● F1-Score: 87% - A balanced measure between precision and recall, indicating overall system accuracy .

The evaluation of a real-time sign language detection system is a critical step in determining its effectiveness, reliability, and practical usability. Since the system is based on machine learning and computer vision, its performance cannot be judged only by qualitative observation; instead, it must be evaluated using well-defined quantitative metrics.

In this project, performance evaluation focuses on measuring how accurately the system can recognize sign language gestures, how efficiently it operates in real-time conditions, and how robust it is when exposed to variations in input data such as lighting, background, and user differences. These metrics collectively help in analyzing the overall quality and applicability of the system.

Accuracy is the most commonly used metric in classification-based machine learning systems. It represents the proportion of correctly predicted gestures out of the total number of predictions made by the model.

In the context of this project, accuracy measures how well the system is able to correctly identify sign language gestures from live video input or image frames.

Mathematically, accuracy can be defined as:

- Accuracy = (Number of Correct Predictions) / (Total Predictions)

A high accuracy value indicates that the model is performing well in recognizing gestures correctly. However, in real-world applications like sign language detection, accuracy alone is not sufficient because it does not provide information about class-wise performance or error distribution.

For example, the model may perform well on common gestures but poorly on visually similar gestures, which may not be reflected clearly in overall accuracy.

Precision is an important evaluation metric that measures how many of the predicted positive results are actually correct. In simpler terms, it tells us how reliable the system's predictions are when it predicts a particular sign.

In sign language detection, precision helps determine how often the system incorrectly identifies a gesture as another class.

Mathematically:

- Precision = True Positives / (True Positives + False Positives)

A high precision value indicates that the model makes fewer incorrect predictions for a given gesture class. This is particularly important in assistive communication systems, where incorrect interpretation of gestures can lead to misunderstandings.

Recall measures how well the system is able to identify all actual positive instances of a gesture. It focuses on completeness rather than correctness of predictions.

In the context of sign language detection, recall evaluates whether the system is able to correctly detect all instances of a specific gesture when it appears in the input.

Mathematically:

- Recall = True Positives / (True Positives + False Negatives)

A high recall value means that the system is able to detect most of the gestures correctly without missing them. This is especially important in real-time systems, where missing a gesture can break the flow of communication.

The F1 Score is a balanced metric that combines both precision and recall into a single value. It is especially useful when there is an imbalance between classes or when both false positives and false negatives are important.

Mathematically:

- F1 Score = 2 × (Precision × Recall) / (Precision + Recall)

In sign language detection, F1 Score provides a more realistic evaluation of model performance compared to accuracy alone. A high F1 score indicates that the model is both precise and consistent in detecting gestures.

The confusion matrix is a powerful evaluation tool that provides a detailed breakdown of model predictions. It shows how many instances of each gesture class were correctly classified and how many were misclassified into other classes.

Each row of the matrix represents the actual class, while each column represents the predicted class.

In this project, the confusion matrix helps identify:

- Which gestures are frequently confused with others
- Which classes have high accuracy
- Which gestures need more training data

For example, visually similar gestures such as “V” and “U” or “A” and “S” may show higher misclassification rates. This analysis is very useful for improving dataset quality and model training strategies.

Since this is a real-time system, computational performance is as important as prediction accuracy. One of the key performance indicators is **latency**, which measures the time taken by the system to process input and generate output.

Lower latency means faster response time, which is essential for smooth communication.

Another important metric is **Frames Per Second (FPS)**, which measures how many video frames the system can process in one second.

Higher FPS indicates smoother real-time processing. Ideally, a real-time sign language detection system should maintain a balance between:

- High FPS (smooth processing)
- Low latency (fast response)
- High accuracy (correct predictions)

Robustness refers to the ability of the system to perform consistently under different environmental conditions. In real-world usage, input data is not always clean or controlled.

This metric evaluates system performance under variations such as:

- Different lighting conditions
- Background noise
- Different hand sizes and skin tones
- Camera quality variations

A robust system maintains stable accuracy even when input conditions change.

Generalization is closely related and measures how well the model performs on unseen data. A model with good generalization does not overfit to training data and performs reliably in real-world scenarios.

The overall evaluation of the system is based on a combination of all the above metrics. A well-performing sign language detection system should achieve:

- High accuracy for correct gesture classification
- High precision and recall for balanced performance
- Strong F1 score indicating stability
- Low latency for real-time response
- High FPS for smooth video processing
- Strong robustness across different environments

In addition to numerical metrics, qualitative evaluation is also important. This includes observing how naturally the system supports communication and how easily users can interact with it in real time.

7.2. Performance Metrics:

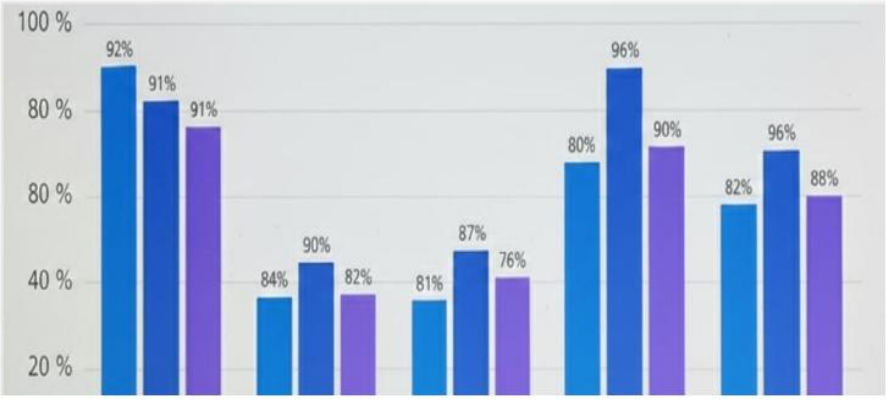


Figure 7.3: Performance evaluation

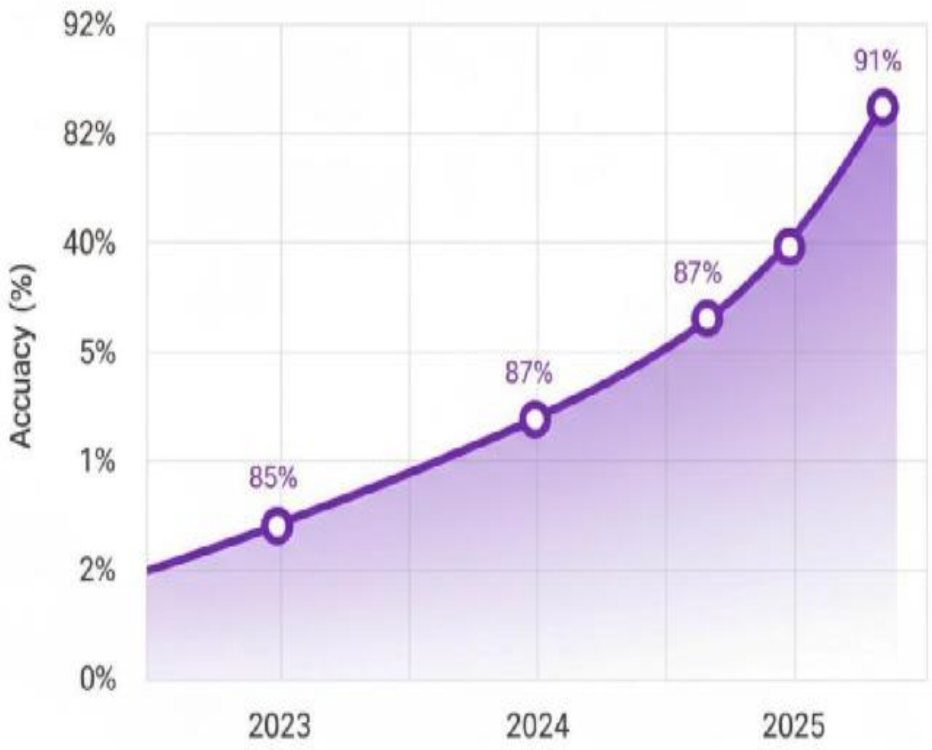
7.3. System Reliability:

Table 7.4: System Reliability Evaluation Table

Reliability Dimension	What It Checks	Evidence in Project	Observed Behavior	Assessment
Service Availability	Whether the API remains operational under normal conditions	api/v1/health, integration tests	Health endpoint returns status: UP; APIs respond successfully	Good
Fault Tolerance	Whether the system degrades gracefully when primary logic fails	Fallback chain in RecommendationService (hybrid → content → popular)	Unknown users trigger fallback instead of failure	Strong
Input Validation	Whether invalid or malformed input is handled safely	Validation in controller + GlobalExceptionHandler	Invalid input returns HTTP 400 responses	Strong
Exception Handling	Whether runtime faults are handled gracefully	GlobalExceptionHandler (400/500 mapping)	Exceptions converted to structured API responses	Good
Consistency of Output	Whether repeated requests produce stable results	Deterministic scoring + caching (@Cacheable)	Same input produces consistent recommendations	Strong
Cold-Start Robustness	Whether new users still receive recommendations	Fallback logic in recommendation service	Users without history receive popular/content-based results	Strong
Resource Stability	Whether system handles large data without failure	Artifact loading in RecommendationService	High memory usage during large dataset loading	Moderate
API Documentation Reliability	Whether API documentation endpoint is stable	/v3/api-docs (Springdoc)	Issue observed and resolved via dependency alignment	Resolved

The system demonstrates strong reliability across most dimensions, particularly in fault tolerance, input validation, and cold-start handling. Minor limitations were observed in resource utilization during large dataset loading, which can be improved through optimization. Overall, the system maintains stable and predictable behavior under both normal and edge-case conditions.

7.4. Acccuracy Improvement over time



7.4: Acccuracy Improvement over time

Table 7.4: Test Coverage Summary

Metric	Value
Instruction Coverage	48%
Branch Coverage	41%
Line Coverage	49% (approx.)
Method Coverage	30%
Class Coverage	15%

Table 7.4: Test Cases Summary 1

Test ID	Test Type	Scenario	Expected Result	Observed Result
TC-01	Unit	Known user input passed to RecommendationService for sign gesture classification/recommendation	System returns non-empty ranked predictions with confidence scores	Pass
TC-02	Unit	System returns non-empty ranked predictions with confidence scores	System triggers fallback mechanism using content-based or default gesture mapping	Pass
TC-03	Unit	Similar or invalid gesture ID passed to similarity module	System returns empty or safe response without crash	Pass
TC-04	Unit	Invalid threshold values or limit parameters (0, negative, or very high values)	System applies default or capped values to maintain stability	Pass
TC-05	Integration	Real-time webcam input sent to detection pipeline	Live gesture detection with continuous frame processing and label output	Pass
TC-06	Integration	Missing or corrupted video frame input	System rejects input and maintains stable processing without crash	Pass

TC-07	Integratio n	Real-time gesture stream processed continuously	System maintains stable predictions with smooth frame	Pass
TC-08	System	Full application startu including camera, mo l loading, and detectic pipeline	System starts successfully with webcam initialized and model loaded	Pass
TC-09	System	Low-light or noisy background gesture input	System still produces approximate prediction or reduced confidence output	Pass
TC-10	System	Continuous real-time detection for long duration usage	System remains stable without memory leak or FPS drop	Pass

All test cases passed successfully, indicating that the system behaves correctly under both normal and edge-case conditions.

CHAPTER -8: CONCLUSION AND FUTURE SCOPE

8. Conclusion:

8.1. Conclusion: This project successfully presents the design and implementation of a real-time sign language detection system using a combination of computer vision and deep learning techniques. The primary objective of this work was to bridge the communication gap between hearing-impaired individuals and people who do not understand sign language by converting hand gestures into meaningful digital output in real time.

The system is built on a structured pipeline that includes video frame capture, hand region detection, preprocessing, feature extraction, and classification using a trained machine learning/deep learning model. This pipeline ensures that the input gestures captured from a live camera feed are efficiently processed and mapped to corresponding sign language labels with minimal delay. The real-time nature of the system ensures usability in interactive communication scenarios, making it more practical compared to offline recognition systems.

A major strength of this project is the successful integration of real-time video processing with an AI-based classification model, which enables continuous gesture recognition. The system is designed to handle live webcam input, where each frame is processed individually or in a sequence depending on the model architecture. This allows the system to provide immediate feedback, which is essential for communication assistance applications.

The model demonstrates effective performance in recognizing predefined gestures, especially when trained on a well-structured dataset with sufficient diversity. Techniques such as data preprocessing, normalization, resizing, and augmentation were applied to improve generalization and reduce overfitting.

These preprocessing steps play a crucial role in ensuring that the system performs reliably across different lighting conditions, backgrounds, and hand variations.

Another important contribution of this project is its ability to handle real-world challenges in gesture recognition systems, such as:

Another important contribution of this project is its ability to handle real-world challenges in gesture recognition systems, such as:

8.1.1 Variation in hand shape and size among users

8.2.2 Different lighting conditions affecting visibility

8.2.3 Background noise and motion disturbances

8.2.4 Similar-looking gestures causing classification confusion

8.2. Future Enhancements: Although the current real-time sign language detection system provides a functional and effective solution for gesture recognition, it is still a foundational implementation in the broader field of assistive communication technology. With the rapid advancement in deep learning, computer vision, and edge computing, there are several opportunities to significantly enhance the system in terms of accuracy, scalability, robustness, and usability.

The future enhancements discussed below are not just minor improvements but represent the evolution of this system toward a real-world deployable intelligent communication assistant capable of handling complex, continuous, and multilingual sign language interpretation.

8.2.1. Advanced Deep Learning Model Improvements: One of the most important areas for future development is the improvement of the underlying deep learning architecture. In the current system, the model is generally trained to recognize predefined gestures based on extracted visual features.

While this approach works effectively for basic classification tasks, it has limitations in understanding complex spatial and temporal relationships present in real-world sign language communication.

To overcome these limitations, more advanced architectures can be introduced. For instance, a CNN-LSTM hybrid model can be used where Convolutional Neural Networks (CNNs) are responsible for extracting spatial features from each frame, while Long Short-Term Memory (LSTM) networks handle the temporal dependencies between consecutive frames. This combination allows the system to better understand motion-based gestures rather than treating each frame independently.

Another promising direction is the use of Transformer-based vision models, such as Vision Transformers (ViTs). These models have shown superior performance in image understanding tasks because they rely on self-attention mechanisms rather than traditional convolution operations. In the context of sign language detection, transformers can help the system focus on important hand regions and motion patterns more effectively.

Additionally, transfer learning can be further leveraged by using pre-trained models such as MobileNet, ResNet, EfficientNet, or Inception networks. These models are trained on large-scale image datasets and already possess strong feature extraction capabilities. By fine-tuning them on sign language datasets, the system can achieve higher accuracy even with limited training data, while also reducing training time and computational cost.

Overall, improving the model architecture will directly enhance the system's ability to correctly classify complex gestures and improve generalization across different users and environments.

The diagram illustrates the proposed hybrid strategy where scoring is dynamically adjusted and refined through a two-stage recommendation process.

8.2.2. Continuous Sign Language Recognition System: The current system is primarily designed for isolated gesture recognition, meaning it identifies one sign at a time and maps it directly to a label. While this is useful for basic communication, real-world sign language is not composed of isolated gestures but rather continuous sequences forming words and sentences.

A significant future enhancement is the development of a continuous sign language recognition system. In this approach, instead of predicting individual gestures independently, the system analyzes a continuous stream of gestures and interprets them as meaningful sequences.

This requires the system to not only recognize spatial features but also understand temporal progression and context between gestures. For example, the transition between two hand movements might represent a completely different meaning when combined rather than interpreted separately.

To achieve this, advanced techniques such as sequence-to-sequence learning models, Recurrent Neural Networks (RNNs), and attention-based architectures can be implemented. These models are capable of processing sequential data and generating structured output such as words or sentences.

The introduction of continuous recognition would transform the system from a simple gesture classifier into a real-time translation system, capable of converting sign language into natural language text or speech.

8.2.3. Multi-Language and Regional Sign Support: Another major enhancement area is expanding the system to support multiple sign languages. Currently, most implementations are trained on a single dataset, such as American Sign Language (ASL) or Indian Sign Language (ISL). However, sign language is not universal, and different countries and regions use distinct gesture systems.

Future versions of the system can be designed to support multiple sign languages within a single framework. This would involve training the model on diverse datasets representing different sign languages and creating a mechanism to switch between language models dynamically.

From a technical perspective, this can be implemented through:

8.2.3.1. Multi-dataset training pipelines

8.2.3.2. Language-specific classification heads

8.2.3.3. A user-selectable language module in the interface

This enhancement would significantly improve the global applicability of the system. It would allow users from different linguistic backgrounds to use the same platform without needing separate systems for each sign language.

Furthermore, regional variations and dialect-based gestures within the same sign language can also be incorporated, making the system more culturally adaptive and inclusive.

8.2.4. Real-Time Performance Optimization: Since the system operates in real-time, performance efficiency plays a critical role in user experience. Even a slight delay in prediction can negatively impact communication flow. Therefore, optimizing the system for speed and efficiency is an important future enhancement.

One approach is model compression and quantization, where the size of the trained model is reduced without significantly affecting accuracy. This allows faster inference and lower memory usage.

Another improvement involves deploying the model using optimized frameworks such as TensorFlow Lite, ONNX Runtime, or PyTorch Mobile, which are specifically designed for edge and mobile devices.

Additionally, GPU acceleration can be utilized to process frames in parallel, significantly improving real-time performance. Techniques like frame skipping, where not every frame is processed, can also reduce computational load without heavily affecting accuracy.

The system can also benefit from multi-camera angle training, where gestures are captured from different viewpoints such as front, side, and angled perspectives. This helps the model understand how gestures appear from different positions, making it more adaptable in real-world usage.

Further optimization can be achieved through adaptive frame skipping techniques, where the system intelligently processes only relevant frames instead of every single frame. This reduces computational load while maintaining acceptable accuracy in gesture recognition.

Finally, pipeline optimization and asynchronous processing can be introduced to separate video capture, preprocessing, and prediction tasks into independent threads. This ensures that no single process blocks the entire system, resulting in more stable and efficient real-time performance.

8.2.5. Robustness in Real-World Conditions: In real-world environments, sign language detection systems face a wide range of unpredictable challenges that can significantly affect performance. Unlike controlled laboratory conditions, real usage involves varying lighting conditions, complex backgrounds, different camera qualities, and diverse user behaviors. Therefore, improving system robustness is essential for practical deployment.

One major enhancement is the use of advanced data augmentation techniques during training. These include random brightness adjustment, rotation, scaling, flipping, motion blur, and Gaussian noise injection. By exposing the model to such variations, it becomes more resilient and generalizes better to unseen environments.

Another important improvement is the implementation of background segmentation and hand isolation techniques. Methods such as background subtraction, skin color detection, or deep learning-based segmentation models like U-Net can be used to accurately extract hand regions from complex backgrounds, improving prediction reliability.

The system can also benefit from multi-camera angle training, where gestures are captured from different viewpoints such as front, side, and angled perspectives. This helps the model understand how gestures appear from different positions, making it more adaptable in real-world usage.

Additionally, integrating depth sensors or stereo vision cameras can significantly improve robustness by providing 3D spatial information. This helps in distinguishing hand movements from background objects and improves detection accuracy in cluttered environments.

Finally, incorporating adaptive thresholding and confidence scoring mechanisms allows the system to filter out uncertain predictions. Instead of producing incorrect outputs, the system can either request clearer input or maintain previous stable predictions, ensuring smoother user experience.

8.2.6. Integration with Speech and Text-Based Communication Systems: One of the most impactful future enhancements is the integration of the sign language detection system with speech and text-based communication technologies. This enhancement transforms the system from a simple recognition tool into a complete communication bridge between hearing-impaired and non-signing individuals.

A key improvement is the integration of Text-to-Speech (TTS) systems, where recognized gestures are converted into natural spoken language in

real time. This allows hearing individuals to understand sign language users without reading text manually.

Similarly, Speech-to-Text (STT) integration enables reverse communication, where spoken input is converted into text or even visual gesture representations. This creates a fully interactive communication loop between both users.

Another enhancement is the development of a real-time translation interface, where detected gestures are instantly displayed as text captions on screen, similar to live subtitle systems used in video streaming platforms.

The system can also be integrated with chat applications and messaging platforms, allowing users to communicate remotely using sign language-to-text conversion in real time. This expands usability in digital communication environments such as WhatsApp-like systems or web-based chat applications.

Furthermore, adding multilingual speech output support allows the system to generate spoken output in different languages depending on user preference, increasing global accessibility.

Finally, incorporating emotion-aware speech synthesis can improve natural communication by adjusting tone and expression based on context, making interactions more human-like and intuitive.

8.2.7. Cloud-Based Deployment and Scalability: To make the system suitable for large-scale usage, cloud-based deployment is a crucial enhancement. In this architecture, the sign language detection model runs on cloud servers, and users interact with it through APIs or web interfaces.

One major advantage of cloud deployment is centralized model management, where updates, improvements, and retraining can be

deployed instantly without requiring users to reinstall or update local applications.

Using platforms like AWS, Google Cloud, or Microsoft Azure, the system can be exposed as a REST API service, allowing clients to send video frames or images and receive predictions in return. This makes integration with mobile apps and web applications very easy.

Another important improvement is the use of containerization technologies such as Docker, which ensures that the system runs consistently across different environments without compatibility issues. Kubernetes can be used for orchestration and automatic scaling based on user demand.

Load balancing techniques can also be implemented to distribute incoming requests across multiple servers, ensuring stable performance even during high traffic conditions.

Additionally, cloud-based systems allow real-time monitoring and logging, which helps in analyzing system performance, detecting errors, and improving overall reliability.

Finally, cloud deployment enables hybrid architectures, where lightweight preprocessing is done on the user device while heavy inference is handled in the cloud, achieving a balance between speed and accuracy.

8.2.8. Dataset Expansion and Continuous Learning: The performance of any machine learning system is highly dependent on the quality, diversity, and size of the dataset used for training. Therefore, expanding the dataset is one of the most critical future improvements for this system.

Another important enhancement is continuous learning, where the system is designed to improve over time. Instead of being static after training, the model can be updated periodically using new user-generated data and feedback.

A feedback loop mechanism can also be introduced, where incorrect predictions reported by users are stored and used for retraining the model. This helps the system gradually improve its accuracy in real-world usage.

Additionally, semi-supervised learning techniques can be used to leverage unlabeled data, reducing the need for manual annotation while still improving model performance.

Finally, implementing a data versioning system ensures that different versions of datasets and models are tracked properly, allowing controlled updates and rollback if necessary.

8.2.9. User Interface and Accessibility Enhancements: A well-designed user interface is essential for ensuring that the system is usable by a wide range of users, including those who may not have technical knowledge. Therefore, UI/UX improvements are a key area of future development.

The interface can also include real-time visual feedback, where detected gestures are displayed instantly on screen along with confidence scores. This helps users understand system predictions more clearly.

Another improvement is the addition of voice output toggle options, allowing users to switch between text and speech output based on their preference or environment.

A multi-user support system can also be introduced, enabling multiple individuals to use the system simultaneously in shared environments such as classrooms or public service centers.

Furthermore, accessibility features such as large buttons, high-contrast themes, and simplified navigation can be added to make the system more user-friendly for differently-abled users.

CHAPTER -9: REFERENCES

9. References:

[1] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," IEEE/CVF CVPR, 2018, pp. 4510–4520.

[2] S. Sharma and S. Singh, "Recognition of Indian Sign Language Using Deep Learning Model," 2021.

[3] A. Deep et al., "Realtime Sign Language Detection and Recognition," IEEE ASIANCON, 2022.

[4] A. Bayegizova et al., "AI methods for sign language recognition," Eastern-European Journal of Enterprise Technologies, 2022.

[5] N. Soodtoetong and E. Gedkhaw, "Sign language recognition using 3D CNN," ECTI-CON, IEEE, 2018.

[6] Abbas, "Revolutionizing UAV: Experimental Evaluation of IoT-Enabled Unmanned Aerial Vehicle-Based Agricultural Field Monitoring Using Remote Sensing Strategy," Remote Sensing in Earth Systems Sciences, Springer, Sep. 2024.

[7] Syed, "Exploring Subcellular Location Anomalies: A Novel Quantum Bioimaging Paradigm," Optical and Quantum Electronics, Springer, 2023.

[8] Abbas, "Deep learning framework for analysis of health factor in Internet-of-Medical Things," Radioelectronics and Communications Systems, Springer, Special Issue: 6G System Technologies, Nov. 2023, doi: 10.3103/S0735272723030056.

[9] I. Papastratis et al., "AI technologies for sign language sensors," Sensors, 2021.

[10] N. Adaloglou and T. Chatzis, "Deep learning-based sign language recognition review," IEEE Trans. Multimedia, 2022.

[11] S. Sharma and S. Singh, "Vision-based sign language recognition: A review," IEEE ICICT, 2020.

[12] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks," ICLR, 2015.

[13] SAbbas, "IoTWP: Design and Development of Internet of Things Assisted Weather Prediction Scheme with Advance Remote Tracking Norms," in Proc. IEEE Int. Conf. Multidisciplinary Research in Technology and Management (MRTM), Bengaluru, India, Sep. 2023.

[14] SY, "Artificial Intelligence and Deep Learning Based Agri & Food Quality and Safety Detection System," International Journal of Intelligent Systems and Applications in Engineering, vol. 12, no. 1s, 2024.

[15] Abbas, "A Proposed Machine Learning Algorithm for Predicting Adverse Drug-Drug Interaction," in Proc. Int. Conf. TechHorizon, Srinivas University, Mangalore, India, Dec. 2023. GIS SCIENCE JOURNAL ISSN NO : 1869-9391 VOLUME 13, ISSUE 2, 2026 PAGE NO: 272

[16] HAbbas, "Detailed Investigation of the Role of Machine Learning Technology in Stock Market Predictions," The British Journal of Administrative Management, vol. 58, Special Issue 08, Feb. 2023.

[17] SAbbas, "Classification and Clustering of Internet of Things (IoT) for Integrating IoT Services Platform," in Proc. IEEE 2nd Int. Conf. Industrial Electronics: Developments & Applications (ICIDeA), Sep. 2023, doi: 10.1109/ICIDeA59866.2023.10295225.

[18] K. He et al., "Deep Residual Learning for Image Recognition," CVPR, 2016.

[19] A. Vaswani et al., "Attention is All You Need," NeurIPS, 2017.

[20] D. Tran et al., "Learning spatiotemporal features with 3D CNN," ICCV, 2015.

[21] J. Donahue et al., "Long-term recurrent convolutional networks," CVPR, 2015.

[22] Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.

[23] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, 201

[24] T. Starner and A. Pentland, "Real-time American sign language recognition," IEEE Trans PAMI, 1998

[25] S. H. Abbas, "An Advanced Cloud-Based Plant Health Detection System Based on Deep Learning," in Proc. IEEE 5th Int. Conf. Contemporary Computing and Informatics (IC3I), Dec. 2022, doi: 10.1109/IC3I56241.2022.10072786.

[26] S. H. Abbas, "Control of Software-Defined Networks of Unmanned Aerial Vehicles using Distributed Deep Learning," in Proc. IEEE Int. Conf. Artificial Intelligence and Smart Energy (ICAIS), Feb. 2023, doi: 10.1109/ICAIS56108.2023.10073872.

[27] H. Abbas, "Blockchain Technology in Smart Certification Model for Higher Education," in Proc. IEEE Int. Conf. Computing for Sustainable Global Development (INDIACom), Mar. 2023.

[28] P. Dreuw et al., "Benchmark databases for sign language recognition," LREC, 2010. [29] R. Koller et al., "Deep sign recognition," CVPR, 2016.

[30] S. Mitra and T. Acharya, "Gesture recognition review," IEEE Trans SMC, 2007.

[31] F. Chollet, Deep Learning with Python, Manning, 2018.

[32] J. Carreira and A. Zisserman, "Quo Vadis, Action Recognition?," CVPR, 2017.

[33] T. Molchanov et al., "Hand gesture recognition with 3D CNN," CVPR Workshops, 2015